

1. Precios (cod_art, fecha_desde, precio).

Escriba una consulta que devuelva el precio de todos los artículos el 13 de mayo de 2021 (devolviendo el código de artículo y el precio).

Opción I:

```
SELECT p.cod_art, p.precio
FROM Precios p
WHERE p.fecha_desde = (
    SELECT MAX(p.fecha_desde)
    FROM Precios p2
    WHERE p2.fecha_desde <= '13-05-2021'
    AND p.cod_art = p2.cod_art
)
```

Opción II:

```
WITH s AS (
    SELECT
        p.cod_art,
        MAX(p.fecha_desde) AS last_date
    FROM
        Precios p
    WHERE
        p.fecha_desde <= '13-05-2021'
    GROUP BY
        p.cod_art
)
```

```
SELECT
    p.cod_art,
    p.precio
FROM
    precios p2
INNER JOIN
    s ON
        p.cod_art = s.cod_art
    AND
        p.fecha_desde = s.last_date;
```

2. a.

$I1 \rightarrow N3$

$I2 \rightarrow N2$

$I3 \rightarrow N1$

b. Si se cae el nodo 1, tenemos que pasar los nodos que pertenecían a N1 a N3, y aquellos que estaban de “resguardo” en N3 pasan a estar en N2.

3. Se sospecha que algunas personas tienen más de un usuario en la red. Para intentar detectarlas se busca encontrar pares de usuario que ambos hayan trabajado en al menos 3 mismas empresas y tengan al menos 5 conocidos en común. Escriba una consulta en Cypher que devuelva los pares de usuarios que cumplen lo anterior, sin repetir el mismo par.

```
MATCH(u1:User)-[:WORKS_IN]->(c:Company)<-[:WORKS_IN]-(u2:username)
WHERE u2.username > u1.username
WITH u1, u2, COUNT(DISTINCT c) AS cantidad_empresas
WHERE cantidad_empresas >= 3
WITH u1, u2
MATCH(u1)-[:KNOWS]->(u3:User)<-[:KNOWS]-(u2)
WITH u1, u2, COUNT(DISTINCT u3) AS cantidad_conocidos
WHERE cantidad_conocidos >= 5
RETURN u1, u2
```

4.

```

01 (BEGIN, T1);
02 (BEGIN, T2);
03 (WRITE, T1, A, 10, 14);
04 (WRITE, T2, B, 5, 8);
05 (COMMIT, T1);
06 (BEGIN CKPT, T2);
07 (BEGIN, T3);
08 (WRITE, T3, D, 18, 20);
09 (WRITE, T2, C, 22, 50);
10 (COMMIT, T2);
11 (END CKPT);
12 (BEGIN, T4);
13 (WRITE, T4, A, 14, 30);
14 (WRITE, T4, B, 8, 10);

```

Primero hago UNDO de abajo hacia arriba

Veo a T4 sin commit -> voy a terminar abortando

B -> 8

A -> 14

Veo commit de T2, ignoro lo que vea de ella

Veo a T3 sin commit -> voy a terminar abortando

D -> 18

Veo el inicio del ckpt, no miro más arriba, y T2 de por sí ya commiteo asique mucho no me servía tampoco.

Hago REDO ahora de aquellas transacciones que hayan commiteado a partir del CHKPT (solamente T2) ->

C -> 50

Se deberá retroceder entonces hasta la línea 06 del log. Se deberá hacer flush a disco de todo tras la recuperación, y en el log se deben sumar además

(ABORT, T3)

(ABORT, T4)

5.

- Clientes (DNI, apellido, nombre, domicilio, telefono)
- Préstamos Vencidos (cod_prestamo, concepto, monto, DNI)

```

SELECT
    c.*
FROM
    Clientes c
    INNER JOIN
        Prestamos_vencidos p
        ON c.DNI = p.DNI

```

Busco junta por único índice, tendría que analizar cual de los dos me conviene más. Veo que el índice de préstamos vencidos cuesta menos, y el resto de números me hacen creerlos, además que es de clustering.

$$Costo(c \bowtie p) = B(c) + n(c) \cdot [Height(I(DNI, p)) + \lceil \frac{B(p)}{V(DNI, p)} \rceil]$$

$$Costo(c \bowtie p) = 500.000 + 5.000.000 \cdot [2 + \lceil \frac{1.000}{5.000} \rceil] = 500.000 + 5.000.000 \cdot [2 + 1] = 15.500.000$$

Veo un costo alto asique pruebo al revés, porque ya de por sí sort merge sería mejor porque sería

$$3 \cdot (B(c) + B(p))$$

Calculo entonces

$$Costo(c \bowtie p) = B(p) + n(p) \cdot [Height(I(DNI, c)) + \lceil \frac{n(c)}{V(DNI, c)} \rceil]$$

$$Costo(c \bowtie p) = 1.000 + 10.000 \cdot [4 + \lceil \frac{5.000.000}{5.000.000} \rceil] = 1.000 + 10.000 \cdot [4 + 1] = 1.000 + 10.000 \cdot 5 = 51.000$$

Veo que es mejor que cualquier otro metodo porque es menor a $B(c) + B(p)$

6. a.

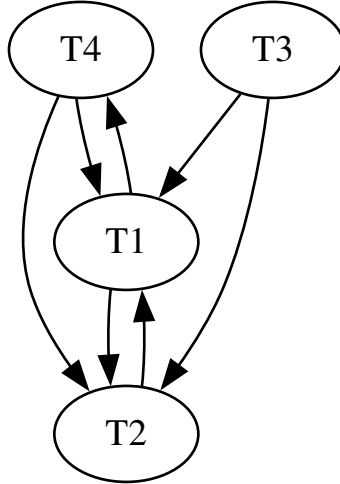


Figure 1: Grafo de precedencias

- b. El solapamiento no es serializable porque el grafo de precedencias no es un DAG
- c. Para que el solapamiento sea recuperable, quiero que las transacciones que hayan leído un dato modificado, commiten después que las transacciones que modificaron.

En la primera lectura, $T3$ lee Y y a nadie la modificó antes, no va generar un problema.

En la segunda lectura $T1$ lee Y que fue modificado por $T2$, pero como $T2$ commita antes que $T1$, tampoco hay un problema aca.

Finalmente en la tercer lectura $T4$ lee X , que fue modificado por la propia $T4$, no genera problemas; fue modificado por $T1$ y $T2$ también y ambas commitan antes que $T4$, por lo cual no hay problemas.

Concluimos entonces que el solapamiento es recuperable.

1.

Se investigará a aquellas personas que luego de ingresar a un establecimiento estatal, al menos 3 familiares suyos ingresaron al mismo establecimiento. Se debe considerar cualquier tipo de parentesco de consanguinidad (padres, hijos, hermanos, etc).

Escriba una consulta en Cypher que resuelva dicho requerimiento

```

MATCH(p1:Persona)-[t:TRABAJA_EN]->(e:Establecimiento)
WITH p1, e
MATCH (p1)-[:HIJO_DE*]-(p2:Persona)-[t2:TRABAJA_EN]->(e)
WHERE t1.ingreso < t2.ingreso
WITH p1, p2, COUNT(DISTINCT p2) AS cantidad_familiares
WHERE cantidad_familiares >= 3
RETURN p1

```

2.

a.

- b. El solapamiento es serializable porque no se forma un DAG.

- c. Para el que el solapamiento sea recuperable las transacciones que lean un dato previamente modificado, deben commitar después que las transacciones que cambiaron dicho dato.

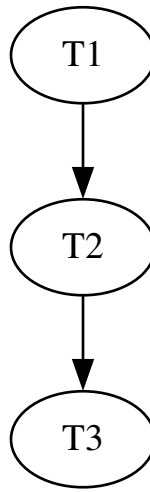


Figure 2: Grafo de precedencias

Busco entonces las lecturas:

$R_{T_2}(Y)$: antes Y fue modificado por $T1 \Rightarrow T1$ tiene que commitear antes que $T2$

$R_{T_3}(Z)$: nadie modificó antes a Z

Entonces con agregar al final algunas de estas opciones:

$C_{T_1}, C_{T_2}, C_{T_3}$

$C_{T_3}, C_{T_1}, C_{T_2}$

$C_{T_1}, C_{T_3}, C_{T_2}$

aseguramos que el solapamiento es recuperable.

3. Como no hay una selección previa ni nada que me pueda hacer descartar datos antes lo único que puedo hacer es
El costo da entonces

$$\text{Costo}(\text{Partidos} \bowtie \text{Partidos}) = 3 \cdot (B(P) \cdot 2) = 6 \cdot B(P) = 6000$$

Calculando la minima cantidad de bloques

$$\frac{B(p)}{M-1} = M-2 \rightarrow B(P) = (M-2) \cdot (M-1) = M^2 - M - 2M + 2 = M^2 - 3M + 2$$

$$M_1 = -76,48; M_2 = 77,007$$

M_1 lo descarto por ser un numero negativo, me quedo con M_2 y redondeo hacia arriba porque voy a necesitar 77 bloques y parte de un 78° bloque, asique respuesta final

$$M = 78$$

.

4.

- Platos (codigo_plato, nombre, precio)
- Ingredientes (id_ingrediente, nombre, tipo, unidad_medida)
- Composición (codigo_plato, id_ingrediente, cantidad)

Escriba una consulta que devuelva el nombre y precio de todos los platos que sean apto veganos. Los veganos únicamente consumen platos que no tengan ingredientes ni de tipo “animal” ni de tipo “derivado de animal”.

```

SELECT DISTINCT(p.nombre), p.precio
FROM Platos p
WHERE NOT EXISTS (
  SELECT 1

```

```

FROM Composicion c
JOIN Ingredientes i ON c.id_ingrediente = i.id_ingrediente
WHERE c.codigo_plato = p.codigo_plato
AND (i.tipo = 'Animal' OR
      i.tipo = 'Derivado de animal')
)

```

5.

Escriba una consulta en MongoDB que encuentre el nombre de museo en el que se encuentre en exhibición el Tyrannosaurus Rex con el molar superior real (el tipo de pieza indica si es real o es una réplica de otro material) más largo del país.

```

db.museos.aggregate([
  {
    $unwind: "$piezas"
  },
  {
    $match: {
      especie: "Tyrannosaurus Rex",
      "piezas.tipo_pieza": "real"
    }
  },
  {
    $sort: {
      "piezas.largo_cm": -1
    }
  },
  {
    $limit: 1
  },
  {
    $project: {
      _id: 0,
      nombre_museo: 1,
      especie: 1,
      pieza: "$piezas.nombre",
      largo_cm: "$piezas.largo_cm",
      tipo_pieza: "$piezas.tipo_pieza"
    }
  }
])

```

6.

```

01 (BEGIN, T1);
02 (WRITE T1, A, 10);
03 (BEGIN, T2);
04 (WRITE T2, B, 5);
05 (COMMIT, T1);
06 (BEGIN, T3);
07 (WRITE T3, A, 20);
08 (BEGIN CKPT, T2, T3);
09 (BEGIN, T4);
10 (WRITE T4, C, 30);
11 (END CKPT);
12 (COMMIT, T2);
13 (WRITE T4, B, 44);
14 (COMMIT, T4);

```

REDO miro de atrás hacia adelante y rehago todas las transacciones que commitearon

Primero veo que las que no commitearon es únicamente T3

Veo un Begin CKPT que tiene un END CKPT por lo cual vuelvo al begin de la transacción más vieja del ckpt (T2)

Desde la línea 3 entonces hago REDO solo de las transacciones que commitearon:

B <- 5 C <- 30 B <- 44

Mando todo a disco, y como T3 no committeo agrego al final del log

(ABORT, T3)

1.

- Envio (id, fecha, hora, monto, cod_repartidor, direccion, barrio)
- Repartidores (cod_repartidor, nombre, fecha_ingreso)

Escriba una consulta que devuelva el código y nombre de los repartidores que efectuaron envíos al barrio de Palermo y nunca efectuaron envíos al barrio de Recoleta ni al de Belgrano

Opción I:

```
SELECT r.cod_repartidor, r.nombre
FROM Repartidores r
     INNER JOIN
           Envios e
           ON r.cod_repartidor = e.cod_repartidor
WHERE e.barrio = 'Palermo'
     AND r.cod_repartidor NOT IN (
           SELECT e2.cod_repartidor
           FROM Envios e2
           WHERE e2.barrio = 'Recoleta' OR e2.barrio = 'Belgrano'
     )
```

Opción II:

```
SELECT r.cod_repartidor, r.nombre
FROM Repartidores r
     INNER JOIN
           Envios e
           ON r.cod_repartidor = e.cod_repartidor
WHERE e.barrio = 'Palermo'
```

EXCEPT

```
SELECT r.cod_repartidor, r.nombre
FROM Repartidores r
     INNER JOIN
           Envios e
           ON r.cod_repartidor = e.cod_repartidor
WHERE e.barrio in ('Recoleta', 'Belgrano')
```

2.

SELECT * FROM R INNER JOIN S ON (R.a = S.a AND S.a >= 2)

Estimo valor de 1 para S.a:

$$n(\sigma_{A=1}(S)) = \lceil \frac{n(S)}{V(A, S)} \rceil = \lceil \frac{5000 - 1000 - 700 - 500}{23 - 3} \rceil$$

$$n(\sigma_{A=4}(R)) = \lceil \frac{2800}{20} \rceil = \lceil 140 \rceil = 140$$

Estimo valor de 4 para R.a:

$$n(\sigma_{A=4}(R)) = \lceil \frac{n(R)}{V(A, R)} \rceil = \lceil \frac{8000 - 1000 - 700 - 500}{23 - 3} \rceil$$

$$n(\sigma_{A=4}(R)) = \lceil \frac{5800}{20} \rceil = \lceil 290 \rceil = 290$$

Luego para el resto de valores, hago la suposición de que los numeros son todos mayores a 1. Por lo cual al momento de hacer la selección en $R.a > 4$ voy a tener $8.000 - 1000 - 700 - 500 - 290 = 5.510$ y en $S.a > 4$ voy a tener $5.000 - 140 - 1.000 - 700 - 500 = 2.660$

Estimo la cardinalidad como, la suma de los $[R.a = S.a = 2] + [R.a = S.a = 3] + [R.a = S.a = 4] + [R.a = S.a > 4]$

$$n(S \bowtie R) = [R.a = S.a = 2] + [R.a = S.a = 3] + [R.a = S.a = 4] + [R.a = S.a > 4]$$

$$n(S \bowtie R) = 700 \cdot 1000 + 500 \cdot 700 + 290 \cdot 500 + 5.510 \cdot 2.660$$

$$n(S \bowtie R) = 15.851.600$$

3.

a.

Para que el solapamiento sea recuperable, las transacciones que leen datos modificados previamente por otra transacción, deben commitear posterior a la transacción que modificaron el dato.

Busco entonces las lecturas:

$R_{T_1}(X)$: nadie realizó ninguna operación antes, no genera problemas.

$R_{T_3}(X)$: antes T_2 escribió en $X \rightarrow T_2$ tiene que commitear antes que T_3 .

$R_{T_3}(Z)$: antes T_1 escribió en $Z \rightarrow T_1$ tiene que commitear antes que T_3 .

Entonces una forma de agregar los commits de forma tal que el solapamiento sea recuperable es agregando al final:

$$C_{T_1}, C_{T_2}, C_{T_3}$$

b. Para que no sea recuperable basta con que T_3 commitee antes que T_1 o T_2 (o antes que ambas). Propongo entonces

$$C_{T_3}, C_{T_1}, C_{T_2}$$

c. Para que el solapamiento sea serializable el grafo de precedencias NO debe ser un DAG.

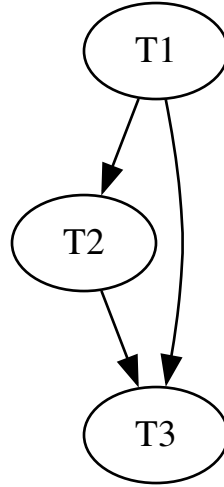


Figure 3: Grafo de precedencias

No se forma un DAG por lo cual el solapamiento es serializable.

4.

Escriba una consulta en MongoDB que devuelva el código de aquellas provincias que en 2024 tuvieron al menos 3 enfermedades con más de 50 casos cada una.

```

db.enfermedades.aggregate([
  {
    $match: {
      "año": 2024
    }
  },
  {
    $unwind: "$casos"
  },
  {
    $group: {
      _id: {
        provincia: "$cod_provincia"
        enfermedad: "$casos.enfermedad"

      },
      cantidad_casos: {
        $sum: 1
      }
    }
  }, {
    $match: {
      cantidad_casos: {$gt: 50}
    }
  },
  {
    $group: {
      _id: "$id.provincia",
      cant_enfermedades: {
        $sum: 1
      }
    }
  },
  {
    $match: {
      cant_enfermedades: {$gte: 3}
    }
  },
  {
    $project: {
      _id: 0,
      cod_provincia: "$_id"
    }
  }
])

```

5.

Mariano quiere encontrar un par de canciones que en conjunto tengan la mayor cantidad de jugadores que le dieron “me gusta”. Es importante que si a un jugador le gustan ambas canciones no debe contarse dos veces sino una. Y finalmente, pero no por ello menos importante, quiere que para ninguna de las dos canciones haya un jugador que no la soporte.

```

MATCH(c1:Cancion), (c2:Cancion)
WHERE (c1.nombre < c2.nombre OR
      (c1.nombre=c2.nombre AND c1.banda < c2.banda))
AND NOT EXISTS {
  MATCH(:Jugador)-[:NO_SOPORTA]->(c1)
} AND NOT EXISTS {
  MATCH(:Jugador)-[:NO_SOPORTA]->(c2)
}
WITH c1, c2
MATCH (j:Jugador)
WHERE (j)-[:LE_GUSTA]->(c1) OR (j)-[:LE_GUSTA]->(c2)

```



```

WITH c1, c2, collect(DISTINCT j) AS jugadores
RETURN c1, c2
ORDER BY size(jugadores) DESC
LIMIT 1

```

6.

```

01 (BEGIN, T1);
02 (WRITE T1, A, 1);
03 (COMMIT, T1);
04 (BEGIN, T2);
05 (WRITE T2, B, 5);
06 (BEGIN, T3);
07 (BEGIN CKPT, T2, T3);
08 (WRITE T3, C, 10);
09 (COMMIT, T3);
10 (BEGIN, T4);
11 (WRITE T4, D, 15);
12 (END CKPT);
13 (COMMIT, T4);

```

Tenemos un checkpoint con su end ckpt por lo cual sabemos que todo lo modificado previo al BEGIN CKPT fueron flusheados a disco.

Por lo cual en disco tenemos:

A <- 1

Luego vemos que C y D fueron escritos por T_3 y T_4 y que han commiteado ambas, pero de todas formas, no tenemos seguridad de que hayan sido flusheados a disco. Como el sistema produjo una falla, tenemos que pueden en disco estar C con valor 0 (inicial) o 10 (escrito por T_3), D con valor 0 (inicial) o 15 (escrito por T_4). Finalmente como de T_2 no tenemos COMMIT, la vamos a tener que abortar, por lo cual en el valor B va seguir estando el valor 0.

1.

- Campeones (nombre_torneo, año, nombre_campeón)
- Torneos (nombre_torneo, país, ciudad, tipo_torneo)

Escriba una consulta que devuelva el nombre de aquellos tenistas que ganaron por lo menos una vez todos los torneos de tipo “Grand Slam”

```

WITH a AS (
    SELECT COUNT(DISTINCT nombre_torneo) AS torneos_grand_slam
    FROM Torneos
    WHERE tipo_torneo = 'Grand Slam'
)

```

```

SELECT
    c.nombre_campeon
FROM
    Campeones c
    INNER JOIN
        Torneos t
        ON c.nombre_torneo = t.nombre_torneo
    CROSS JOIN
        a
WHERE
    tipo_torneo = 'Grand Slam'
GROUP BY
    c.nombre_campeon
HAVING
    COUNT(DISTINCT c.nombre_torneo) >= a.torneos_grand_slam

```

2.

Datos:

N1: 59

N2: 160

N3: 120

N4: 235

Respuesta:

a.

I1: 100 -> va al nodo N3

I2: 130 -> va al nodo N2

I3: 240 -> va al nodo N1

b.

Si se cae el nodo N1 todo lo que estaba en él va pasar a estar en N3, y todo lo que estaba guardado de “resguardo” en N3, que realmente pertenecía a N1, se pasa a N2.

3.

Escriba una consulta en MongoDB que devuelva el código y nombre de aquellas provincias que no tengan ningún hospital con un promedio de encuestas menor a 5.

```
db.encuestas.aggregate([
  {
    $unwind: "$encuestas"
  },
  {
    $group: {
      _id: {
        cod_provincia: "$cod_provincia",
        provincia: "$provincia",
        nombre_hospital: "$nombre_hospital",
        direccion: "$direccion"
        // asumo que puedo tener más de un hospital con el mismo nombre en la misma provincia, uso todo
      },
      promedio_hospital: {$avg: "$encuestas.puntaje"}
    }
  }, // obtengo el promedio por hospital primero
  {
    $group: {
      _id: {
        cod_provincia: "$_id.cod_provincia",
        nombre_provincia: "$_id.provincia"
      },
      cant_hospitales_promedio: {
        $sum: {
          $cond: [ { $lt: [ "$promedio_hospital", 5 ] }, 1, 0 ]
        }
      }
    }
  }, // agrupo por provincia y voy contando la cantidad de hospitales con promedio menor a 5 que hay en esa
  {
    $match: {
      "cant_hospitales_promedio": {
        $eq: 0
      }
    }
  }, // descarto aquellos hospitales que tienen algun hospital con promedio menor a 5
  {
    $project: {
      _id: 0,
      cod_provincia: "$_id.cod_provincia",
      provincia: "$_id.provincia"
    }
  }
] // proyecto únicamente los datos que me importan, capaz solo mostrando el id es lo mismo, idk
```

1)

4.

$$b_{T1}; R_{T1}(X); b_{T2}; W_{T2}(Y); b_{T3}; R_{T3}(X); W_{T3}(X); R_{T3}(Y)$$

a.

Para que el solapamiento sea recuperable las transacciones que lean un dato modificado previamente por otra transacción, deben commitear después que la transacción “modificadora”.

Busco lecturas entonces:

$R_{T1}(X)$: Nadie realiza ninguna operación antes, no genera ningún problema.

$R_{T3}(X)$: Solo T_1 realizó una lectura antes, no genera ningún problema tampoco.

$R_{T3}(Y)$: La transacción T_2 modificó a $Y \rightarrow T_2$ debe commitear antes que T_3

Por lo cual cualquier orden de commits en los que el commit de T_2 anteceda al commit de T_3 ; propongo uno cualquiera que cumpla con esto

$$C_{T1}, C_{T2}, C_{T3}$$

- b. Acá podría probar todas las combinaciones posibles de lectura/escritura por transacción y por ítem pero prefiero directamente armar el grafo de precedencias, y mirar qué aristas debería agregar para formar un DAG, logrando que el solapamiento **deje** de ser serializable.

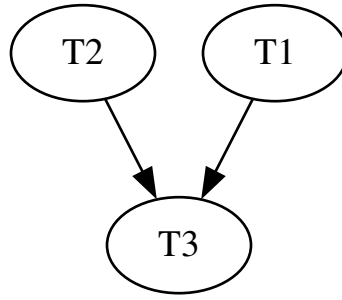


Figure 4: Grafo de precedencias

Como se puede ver de agregar conflictos entre T2 y T1 no puedo agregar una única lectura o escritura que genere que el grafo sea un DAG, por lo cual podría agregar al final:

Con ítem Y:

- $R_{T1}(Y)$: Agrega arista de T2 a T1, **sigue siendo serializable**.
- $R_{T2}(Y)$: No agrega ninguna arista pues quien modifica a Y es T2 misma, **sigue siendo serializable**.
- $R_{T3}(Y)$: No agrega ninguna arista pues ya existe de T2 a T3 por la lectura, **sigue siendo serializable**.
- $W_{T1}(Y)$: Agrega arista de T2 a T1, **sigue siendo serializable**.
- $W_{T2}(Y)$: Agrega una arista de T3 a T2 $\rightarrow$ se forma un ciclo $\rightarrow$ **deja de ser serializable**.
- $W_{T3}(Y)$: No agrega ninguna arista pues ya existe de T2 a T3 por la lectura, **sigue siendo serializable**.

Con ítem X:

- $R_{T1}(X)$: Agrega una arista de T3 a T1 $\rightarrow$ se forma un ciclo $\rightarrow$ **deja de ser serializable**.
- $R_{T2}(X)$: Agrega una arista de T3 a T2 $\rightarrow$ se forma un ciclo $\rightarrow$ **deja de ser serializable**.
- $R_{T3}(X)$: No agrega ninguna arista pues quien modifica a X es T3 misma, **sigue siendo serializable**.
- $W_{T1}(X)$: Agrega una arista de T3 a T1 $\rightarrow$ se forma un ciclo $\rightarrow$ **deja de ser serializable**.
- $W_{T2}(X)$: Agrega una arista de T3 a T2 $\rightarrow$ se forma un ciclo $\rightarrow$ **deja de ser serializable**.
- $W_{T3}(X)$: No agrega ninguna arista, **sigue siendo serializable**.

5.

- listadoempresas (id_empresa, nombre, nombre_contacto, ciudad)
- vendedores (legajo, nombre, celular, ciudad, fecha_ingreso)

SELECT * FROM listadoempresas INNER JOIN vendedores USING(ciudad);

Tengo $M = 12$.

No puedo aplicar junta por índices, voy a tener que buscar con los otros métodos de junta.

Busco descartar los métodos con restricciones primero:

Para sort merge se tiene que cumplir que:

$$M \geq \lceil \frac{B(R)}{V(A, R)} \rceil + \lceil \frac{B(S)}{V(A, S)} \rceil + 1$$

Reemplazando con los datos

$$12 \geq \lceil \frac{1.000}{100} \rceil + \lceil \frac{2.000}{50} \rceil + 1$$

$$12 \geq \lceil 10 \rceil + \lceil 40 \rceil + 1 \rightarrow 12 < 51$$

No se cumple la restricción, no puedo ordenar en memoria, busco costo ordenado con sort externo

$$Cost(Empresas \bowtie Vendedores) = Cost(Ord_M(R)) + Cost(Ord_M(S)) + B(R) + B(S)$$

$$Cost(Empresas \bowtie Vendedores) = 2 \cdot B(R) \cdot \lceil \log_{M-1}(B(R)) \rceil + 2 \cdot B(S) \cdot \lceil \log_{M-1}(B(S)) \rceil + B(R) + B(S)$$

$$Cost(Empresas \bowtie Vendedores) = 2 \cdot 1.000 \cdot \lceil \log_{11}(1.000) \rceil + 2 \cdot 2.000 \cdot \lceil \log_{11}(2.000) \rceil + 1.000 + 2.000$$

$$Cost(Empresas \bowtie Vendedores) = 2.000 \cdot \lceil 2,88 \rceil + 4.000 \cdot \lceil 3,16 \rceil + 1.000 + 2.000$$

$$Cost(Empresas \bowtie Vendedores) = 2.000 \cdot 3 + 4.000 \cdot 4 + 1.000 + 2.000 = 25.000$$

Busco las restricciones de junta hash-grace:

$$P \leq M - 1 \wedge \min(\lceil \frac{B(R)}{P} \rceil; \lceil \frac{B(S)}{P} \rceil) \leq M - 2$$

Como $P = M - 1$ el menor entre las dos divisiones será el de $B(listadoEmpresas)$. Reemplazo los datos

$$\lceil \frac{1.000}{M-1} \rceil \leq M - 2$$

$$\lceil \frac{1.000}{11} \rceil \leq 10$$

$$\lceil 90,9 \rceil > 10$$

No puedo aplicar hash grace.

Aplico entonces Loops anidados por bloque:

$$Cost(R \bowtie S) = B(R) + \lceil \frac{B(R)}{M-2} \rceil \cdot B(S)$$

$$Cost(Empresas \bowtie Vendedores) = 1.000 + \lceil \frac{1.000}{10} \rceil \cdot 2.000$$

$$Cost(Empresas \bowtie Vendedores) = 1.000 + \lceil 100 \rceil \cdot 2.000 = 201.000$$

Entonces como veo que el costo de sort merge es mejor, me quedo con él concluyendo entonces:

$$Cost(Empresas \bowtie Vendedores) = 25.000$$

Calculando ahora la cantidad de filas que serán devueltas:

$$n(R \bowtie S) = \frac{n(R) \cdot n(S)}{\max(V(A, R), V(A, S))}$$

$$n(Empresas \bowtie Vendedores) = \frac{10.000 \cdot 50.000}{\max(100, 50)}$$

$$n(Empresas \bowtie Vendedores) = \frac{10.000 \cdot 50.000}{100} = \frac{500.000.000}{100} = 5.000.000$$

6.

```
01 (BEGIN, T1);
02 (WRITE T1, A, 1);
03 (COMMIT, T1);
04 (BEGIN, T2);
05 (WRITE T2, A, 2);
06 (BEGIN, T3);
07 (BEGIN CKPT, T2, T3);
08 (WRITE T3, C, 3);
09 (COMMIT, T3);
10 (BEGIN, T4);
11 (WRITE T4, D, 4);
```

T1 y T3 commitearon, T2 y T4 no. No hay un END CKPT, solo miro hasta 04 porque son las activas en el BEGIN CKPT

```
A <- 2
D <- 4
```

Deshago solo las operaciones de aquellas transacciones que no commitearon (T2 y T4).

Finalmente en el log agrego

```
(ABORT, T2)
(ABORT, T4)
```

y vuelco todo a disco

1.

- Socios (id_socio, nombre, fecha_nacimiento)
- SocioAdoraPelicula(id_socio, nombre_pelicula)

Escriba una consulta que devuelva el nombre de la/s película/s que le gustan tanto al socio mas joven como al socio mas viejo

```
WITH fecha_mas_joven AS (
    SELECT id_socio
    FROM Socios
    WHERE fecha_nacimiento = (
        SELECT MAX(fecha_nacimiento)
        FROM Socios
    )
), fecha_mas_viejo AS (
    SELECT id_socio
    FROM Socios
    WHERE fecha_nacimiento = (
        SELECT MIN(fecha_nacimiento)
        FROM Socios
    )
```

```
)

SELECT nombre_pelicula
FROM SocioAdoraPelicula
WHERE id_socio = (SELECT id_socio FROM fecha_mas_joven)
```

```
INTERSECT
```

```
SELECT nombre_pelicula
FROM SocioAdoraPelicula
WHERE id_socio = (SELECT id_socio FROM fecha_mas_viejo)
```

2.

Escriba una consulta que encuentre cuales son las 5 más seguidas por usuarios que tienen conocimiento en SQL.

```
db.linked_out.aggregate([
  {
    $unwind: "$conocimientos"
  }, // desacoplo conocimientos, voy a tener varias veces al usuario
  {
    $match: {
      conocimientos: "SQL"
    }
  }, // pero como aca solo matcheo con una sola cosa (SQL en este caso), no voy a tener repetidos los following
  {
    $unwind: "$sigueA"
  }, // desacoplo por seguidos para sumar los seguidores
  {
    $group: {
      _id: "$sigueA.cuenta",
      followers: {$sum: 1}
    }
  },
  { // agrupo por cuenta seguida, contando cuantas veces aparece (es decir, cuantos seguidores tiene)
    $sort: {
      followers: -1
    }
  },
  {
    $limit: 5
  },
  {
    $project: {
      _id: 1,
      followers: 1
    }
  }
])
```

3.

Se pide encontrar las bandas que tengan al menos una canción propia con un mismo género musical que alguna canción de la banda "Eruca Sativa".

De todas esas bandas, se deben devolver únicamente las 5 que tengan la mayor cantidad de usuarios a los que les gusta algun tema de esa banda y también algun tema de "Eruca Sativa".

```
MATCH(c1:Cancion)-[:PERTENECE_A]->(b1:Banda{nombre:"Eruca Sativa"}),
      (c1)-[:ES_DE]->(g1:Genero)
WITH c1, g1 // tengo las canciones de Eruca Sativa y todos los géneros que tocó la banda
MATCH(c2:Cancion)-[:PERTENECE_A]->(b2:Banda)
WHERE b2.nombre <> "Eruca Sativa"
WITH c1, g1, c2, b2
MATCH (c2)-[:ES_DE]->(g1) // busco canciones de una banda distinta a 'Eruca Sativa' que se asocie a algún genero
```

```

WITH c1, c2, b2
MATCH (u:Usuario)-[:LE_GUSTA]->(c2),
      (u)-[:LE_GUSTA]->(c1) // me fijo usuarios que les guste canciones de esa banda y que tmb le guste algun tema
WITH b2, COUNT(DISTINCT(u)) AS likes
RETURN b2.nombre AS banda, likes
ORDER BY likes DESC
LIMIT 5

```

4.

```

Reserva: dia, horario, servicio
DFs: dia, horario -> servicio
CCs: dia y horario

```

```

Niños: DNI, nombre, apellido, fecha_nacimiento, alergia, telefono_contacto, nombre_contacto
DFs: DNI -> nombre, apellido, fecha_nacimiento, alergia, telefono_contacto, nombre_contacto
CCs: DNI

```

En principio pensé en tener los telefonos y nombre separado del contacto, pero eso genera que si van dos hermanos a un salón de juego el numero de contacto tenga que ser distinto, y no tiene mucho sentido que sea así

5.

- Papers(id, titulo, fecha_publicacion, ciudad, categoria, contenido)
- AutoresPapers(id_paper, id_autor)
- Autores(id, nombre, apellido, nacionalidad)

```

SELECT *
FROM Papers p
      INNER JOIN AutoresPapers ap ON (p.id = ap.id_paper)
      INNER JOIN Autores a ON (a.id = ap.id_autor)
WHERE p.categroai = 'Tecnología'

```

Primero hago la selección:

Veó que no tengo datos de tecnología, asique asumo distribución uniforme

$$Costo(\sigma_{Categoría='Tecnología'}(Papers)) = B(Papers) = 1.000$$

No tengo índices asique no me queda otra que hacer un file scan.

$$n(\sigma_{Categoría='Tecnología'}(Papers)) = \lceil \frac{n(Papers)}{V(Categoría, Papers)} \rceil$$

$$n(\sigma_{Categoría='Tecnología'}(Papers)) = \lceil \frac{10.000}{100} \rceil = \lceil 100 \rceil = 100$$

Estimo ahora la cantidad de bloques para saber cuantos voy a tener en memoria (para hacer pipelining)

$$B(\sigma_{Categoría='Tecnología'}(Papers)) = \lceil \frac{n(\sigma_{Categoría='Tecnología'}(Papers))}{F(Papers)} \rceil = \lceil \frac{n(\sigma_{Categoría='Tecnología'}(Papers))}{\frac{n(Papers)}{B(Papers)}} \rceil = \lceil \frac{100}{\frac{10.000}{1.000}} \rceil = \lceil 10 \rceil = 10$$

Junta con AutoresPapers. Junta con índice ya lo descarto. Pruebo con Sort Merge

$$M \geq \lceil \frac{B(Papers)}{V(A, R)} \rceil + \lceil \frac{B(AutoresPapers)}{V(id_{paper}, AutoresPapers)} \rceil + 1$$

Respecto a lo primero ya se cuanto vale el B tras la selección, pero como igual dividido por la variabilidad respecto al id del paper, dividido por las rows que me dio la selección, utilizo directamente el valor

$$M \geq \lceil \frac{10}{100} \rceil + \lceil \frac{200}{10.000} \rceil + 1$$

$$1000 \geq 3$$

Puedo usarlo, calculo el costo

$$Cost(Papers \bowtie AutoresPapers) = Cost(Ord_M(Papers)) + Cost(Ord_M(AutoresPapers)) + B(Papers) + B(AutoresPapers)$$

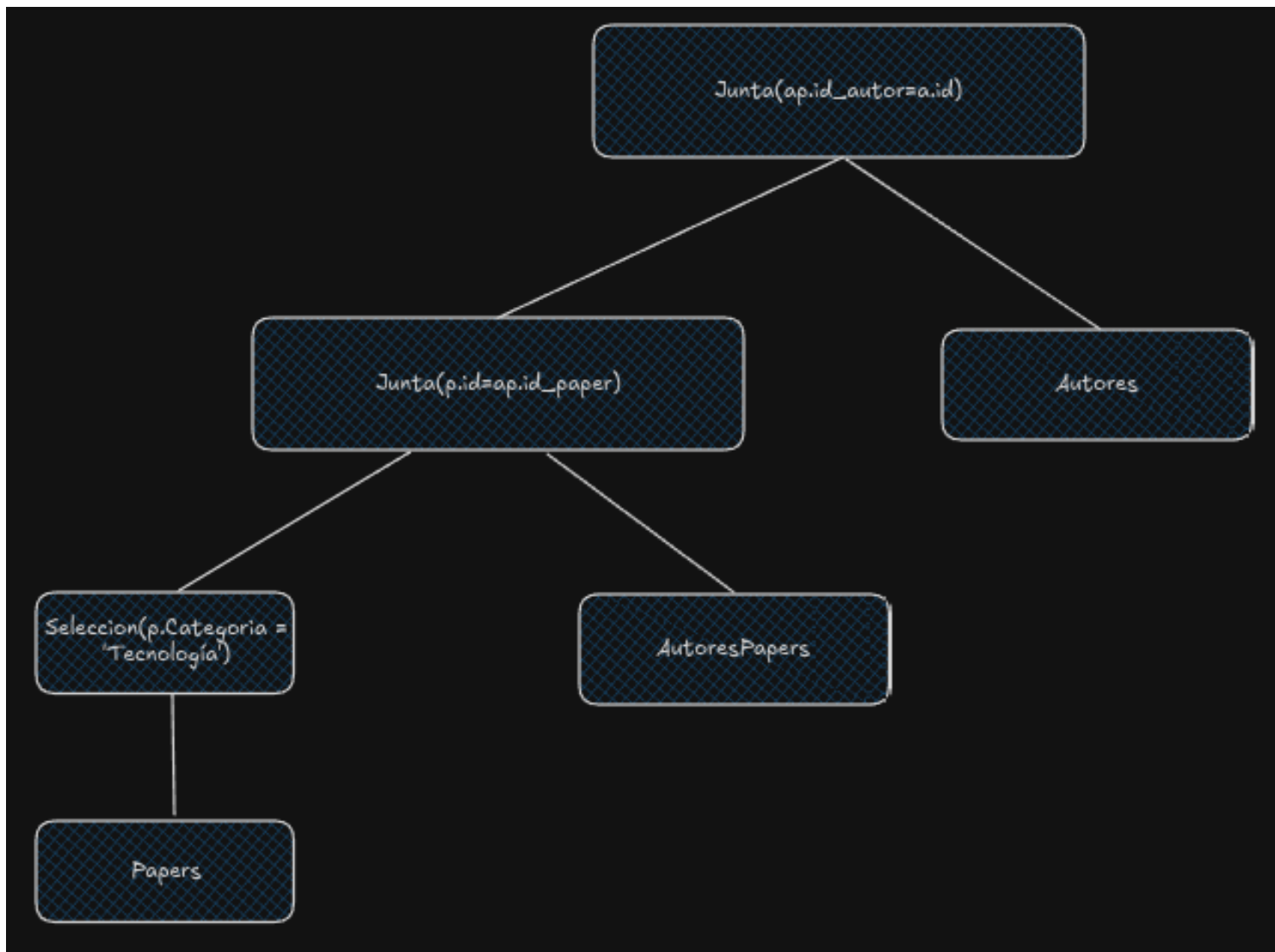


Figure 5: Árbol de consultas

$$Cost(Papers \bowtie AutoresPapers) = 2 \cdot B(Papers) \cdot \lceil \log_{M-1}(B(Papers)) \rceil + 2 \cdot B(AutoresPapers) \cdot \lceil \log_{M-1}(B(AutoresPapers)) \rceil + B(Papers)$$

$$Cost(Papers \bowtie AutoresPapers) = 2 \cdot 10 \cdot \lceil \log_{999}(10) \rceil + 2 \cdot 200 \cdot \lceil \log_{999}(200) \rceil + 10 + 200$$

$$Cost(Papers \bowtie AutoresPapers) = 2 \cdot 10 \cdot \lceil \log_{999}(10) \rceil + 2 \cdot 200 \cdot \lceil \log_{999}(200) \rceil + 10 + 200$$

$$Cost(Papers \bowtie AutoresPapers) = 20 \cdot \lceil 1 \rceil + 400 \cdot \lceil 1 \rceil + 10 + 200 = 20 + 400 + 10 + 200 = 630$$

Busco ahora joinear con hash grace. Veo si cumple las restricciones:

$$P \leq M - 1$$

$$\min(\lceil \frac{B(Papers)}{P} \rceil, \lceil \frac{B(AutoresPapers)}{P} \rceil) \leq M - 2$$

$$\lceil \frac{10}{999} \rceil \leq 998$$

Se cumple, busco costo:

$$Cost(Papers \bowtie AutoresPapers) = 2 \cdot B(Papers) + 3 \cdot B(AutoresPapers) = 2 \cdot 10 + 3 \cdot 200 = 620$$

Calculo ahora con loops anidados:

$$Cost(Papers \bowtie AutoresPapers) = \lceil \frac{B(Papers)}{M-2} \rceil \cdot B(AutoresPapers) = \lceil \frac{10}{998} \rceil \cdot 200 = 200$$

Uso loops anidados

Calculo cardinalidad y bloques de esta junta:

$$n(Papers \bowtie AutoresPapers) = \frac{n(Papers) \cdot n(AutoresPapers)}{\max(V(id_{paper}, Papers), (id_{paper}, AutoresPapers))}$$

$$n(Papers \bowtie AutoresPapers) = \frac{100 \cdot 20.000}{\max(100, 10.000)} = \frac{2.000.000}{10.000} = 200$$

$$F(Papers \bowtie AutoresPapers) = \lfloor \frac{1}{\frac{1}{F(Papers)} + \frac{1}{F(AutoresPapers)}} \rfloor$$

$$F(Papers \bowtie AutoresPapers) = \lfloor \frac{1}{\frac{1.000}{10.000} + \frac{200}{20.000}} \rfloor = \lfloor 9.09 \rfloor = 9$$

$$B(Papers \bowtie AutoresPapers) = \frac{n(Papers \bowtie AutoresPapers)}{F(Papers \bowtie AutoresPapers)} = \frac{200}{9} = 22.22 = 23$$

Puedo tenerlos en memoria también

Hago el join final con Autores, llamo JOIN1 a lo hecho previamente

$$Cost(JOIN1 \bowtie Autores) = \lceil \frac{B(JOIN1)}{M-2} \rceil \cdot B(Autores) = \lceil \frac{23}{998} \rceil \cdot 50 = 50$$

Ya veo a ojo que no voy a encontrar un resultado mejor con otro método de junta.

$$Costo(Papers \bowtie AutoresPapers \bowtie Autores) = Costo(\sigma_{Categoría='Tecnología'}(Papers)) + Costo(Papers \bowtie AutoresPapers) + Costo(JOIN1 \bowtie Autores)$$

$$Costo(Papers \bowtie AutoresPapers \bowtie Autores) = 1.000 + 200 + 50 = 1.250$$

6.

```
01 (BEGIN, T1);
02 (WRITE T1, A, 10);
03 (COMMIT, T1);
04 (BEGIN, T2);
05 (WRITE T2, A, 20);
06 (BEGIN CKPT, T2);
07 (BEGIN, T3);
08 (WRITE T3, B, 30);
09 (COMMIT, T3);
10 (COMMIT, T2);
11 (END CKPT);
10 (BEGIN, T4);
11 (WRITE T4, D, 40);
```

Como veo un begin ckpt y un end ckpt solo voy a mirar desde la transaccion más vieja del begin ckpt (T2). Rehago las transacciones commiteadas y aborto las no commiteadas

Commiteadas: T2 y T3

A <- 20 B <- 30

Aborto las no commiteadas, agregando al log

(ABORT, T4)

Hago flush a disco de todo!

1.

- Socios(id_socio, nombre, fecha_nacimiento)
- Calificaciones(id_socio, nombre_pelicula, estrellas_calificacion)

Escriba una consulta que devuelva el nombre de la/s película/s que han recibido una calificación de 5 estrellas por todos los socios de la asociación.

$$\begin{aligned}ids &= \pi_{id_socio} (Socios) \\calificaciones_cinco &= \sigma_{estrellas_calificacion=5} (Calificaciones) \\peliulas_cinco &= \pi_{nombre_pelicula, id_socio} (calificaciones_cinco) \\respuesta &= peliculas_cinco \div ids \\&\pi_{nombre_pelicula}(respuesta)\end{aligned}$$

2.

Hallar usuarios que le dieron me gusta al menos 80% de los temas que la banda sacó en 2024.

```
MATCH (c1:Cancion{banda:"Mac y los McMaccers", año: 2024})
WITH COUNT(DISTINCT c1) AS canciones_2024
MATCH (u1)-[:LE_GUSTA]->(c2:Cancion{banda:"Mac y los McMaccers", año: 2024})
WITH u1, COUNT(DISTINCT c2) AS canciones_que_gustan, canciones_2024
WHERE canciones_que_gustan >= 0.8 * canciones_2024
RETURN u1
```

3.

```
CREATE COLUMNFAMILY alumnos(
  padron INT,
  nombre TEXT STATIC,
  apellido TEXT STATIC,
  codigo_materia TEXT,
  fecha DATE,
  nota INT,
  PRIMARY KEY ((padron), codigo_materia, fecha)
);
```

- a. Se puede pues estamos buscando por la clave de partición entera
 - b. No se puede pues no estamos buscando por la clave de partición, ni siquiera es parte de la PK el apellido.
 - c. No se puede pues si bien se usa la clave de partición y luego una clave de clustering, se tiene que filtrar p
 - d. Se puede pues se busca por clave de partición + la primera clave de clustering
 - e. Se puede pues se busca por clave de partición + el conjunto de las claves de clustering
- 4.

Primero que nada hago el grafo de precedencias para ver en que estado está el solapamiento antes de agregar algo.

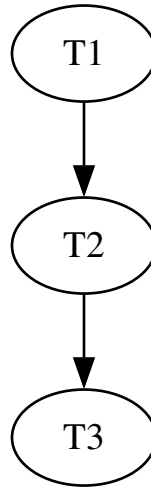


Figure 6: Grafo de precedencias

Recordando que un solapamiento no es serializable cuando su grafo de precedencias es un DAG

- a. $W_{T_1}(X)$. Al agregar esta escritura al final se va generar una arista de T_2 a T_1 generando un ciclo.
- b. $R_{T_2}(Y)$. Al agregar esta lectura al final se va a generar una arista de T_3 a T_2 generando un ciclo.
- c. Para que el solapamiento sea recuperable todas las transacciones que lean un item previamente modificado por otra transacción, deben commitear después que la transaccion “modificadora”. Analizo las lecturas:

$R_{T_1}(X)$: nadie realizó ninguna operación previa asique no genera ningún problema.

$R_{T_2}(X)$: nadie modificó a X antes asique tampoco genera ningún problema.

$R_{T_3}(X)$: T_2 modificó antes a X asique T_2 tiene que commitear antes que T_3 .

$R_{T_3}(Y)$: nadie modificó a Y antes asique tampoco genera ningún problema.

Concluyo entonces que con tal de que T_2 commitee antes que T_3 el solapamiento va ser recuperable. Propongo entonces:

$$C_{T_1}, C_{T_2}, C_{T_3}$$

5.

```

01 (BEGIN, T1);
02 (WRITE T1, A, 10);
03 (BEGIN, T2);
04 (WRITE T1, B, 20);
05 (COMMIT, T1);
06 (WRITE T2, B, 30);
07 (BEGIN CKPT, T2);
08 (BEGIN, T3);
09 (WRITE T3, C, 40);
10 (COMMIT, T3)
11 (BEGIN, T4);
  
```

12 (WRITE T4, D, 50);

Como es el algoritmo de UNDO, tengo que deshacer aquellas operaciones que no commitearon y abortarlas.

Commitearon: T3 (T1 tambien pero no la miro porque ya en el CKPT me daba cuenta que no estaba activa)

No commitearon: T2, T4.

Llego hasta la línea 03 que es donde inicia la transacción mas antigua del CKPT (T2)

D <- 50 B <- 30

Agrego en el log

(ABORT, T2)

(ABORT, T4)

Flusheo todo a disco (log y valores de las “variables”).

6.

- Usuarios(id, nombre_usuario, apodo, fecha_inscripcion)
- Apuestas(id_evento, id_usuario, tipo_apuesta, monto_apostado)
- Eventos(id, fecha, descripcion, tipo)

```
SELECT *  
FROM Usuarios u  
      INNER JOIN Apuestas a ON (u.id = a.id_usuario)  
      INNER JOIN Eventos e ON (a.id_evento = e.id_evento)  
WHERE e.tipo = 'Fútbol Americano';
```

Como no tengo indice alguno para buscar los eventos de tipo, hago un file Scan

$$Cost(\sigma_{tipo='Fútbol Americano'}(Eventos)) = B(E) = 1000$$

Como no hay datos sobre como se distribuyen los eventos según el tipo, estimo que se distribuyen uniformemente, por lo cual en memoria (para hacer el pipelining), voy a tener

$$n(\sigma_{tipo='Fútbol Americano'}(Eventos)) = \frac{n(E)}{V(tipo, Eventos)} = \frac{10.000}{100} = 100$$

Calculando la cantidad de bloques entonces:

$$B(\sigma_{tipo='Fútbol Americano'}(Eventos)) = \lceil \frac{n(\sigma_{tipo='Fútbol Americano'}(Eventos))}{F(R)} \rceil$$

$$B(\sigma_{tipo='Fútbol Americano'}(Eventos)) = \lceil \frac{100}{\frac{10.000}{1.000}} \rceil = \lceil 10 \rceil = 10$$

Calculo ahora el costo de la junta con Apuestas:

No tengo índices asique ya descarto esa posibilidad. Como tengo muchos bloques busco primero con junta por loops anidados por bloque:

$$Cost(Eventos \bowtie Apuestas) = \lceil \frac{B(\sigma_{tipo='Fútbol Americano'}(Eventos))}{M-2} \rceil \cdot B(Apuestas)$$

$$Cost(Eventos \bowtie Apuestas) = \lceil \frac{10}{1.000-2} \rceil \cdot 400 = 400$$

Viendo que me queda solo el costo de $B(Apuestas)$ ya puedo estar seguro que no voy a encontrar por otro método un costo mejor que este.

Ahora calculo la cardinalidad hasta este punto, y la cantidad de bloques:

$$n(Eventos \bowtie Apuestas) = \frac{n(Eventos) \cdot n(Apuestas)}{\max(V(id, Eventos), V(id_evento, Apuestas))}$$

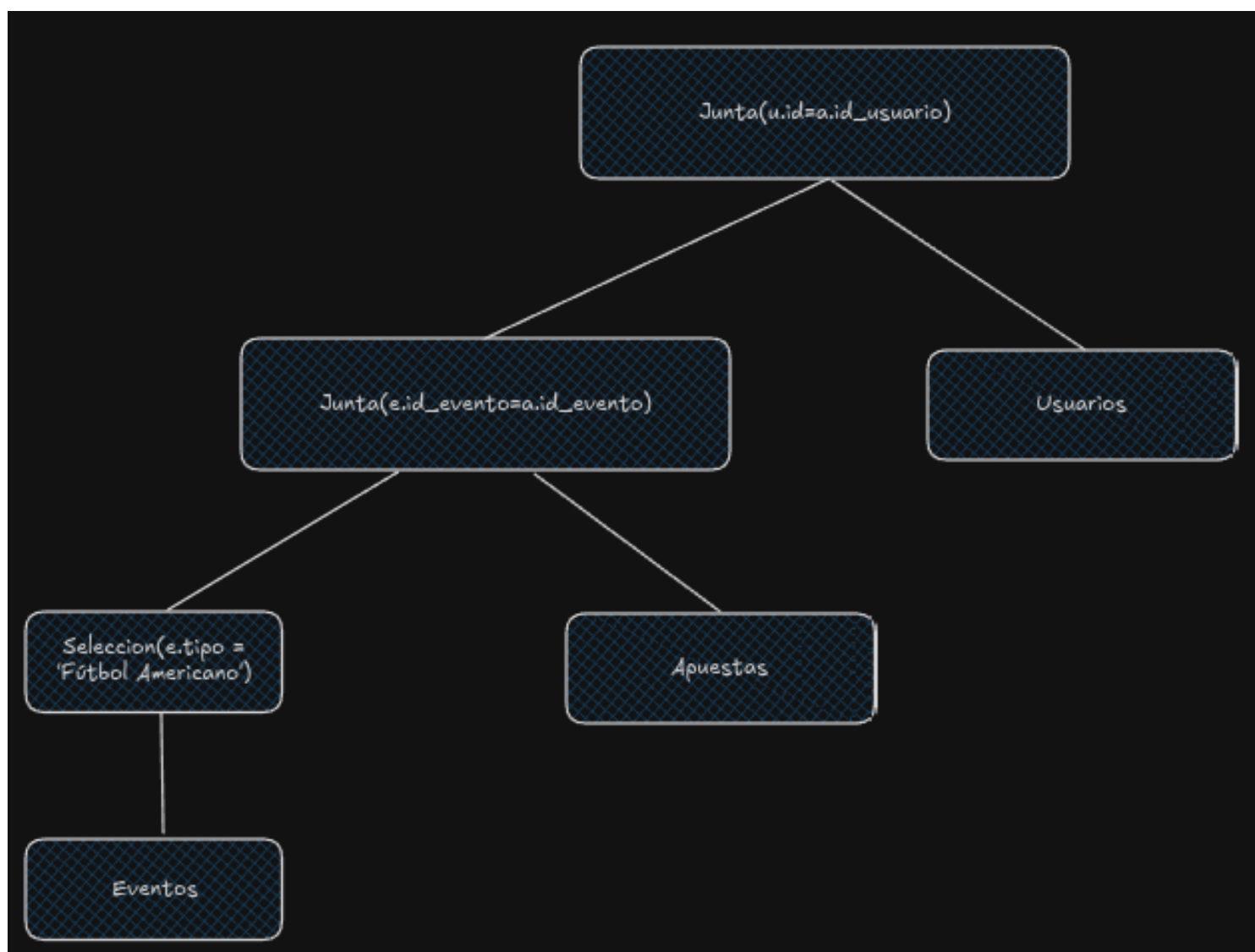


Figure 7: Árbol de consultas

$$n(Eventos \bowtie Apuestas) = \frac{100 \cdot 40.000}{\text{máx}(100, 10.000)} = \frac{4.000.000}{10.000} = 400$$

$$F(Eventos \bowtie Apuestas) = \lceil \frac{1}{\frac{1}{F(Eventos)} + \frac{1}{F(Apuestas)}} \rceil$$

$$F(Eventos \bowtie Apuestas) = \lceil \frac{1}{\frac{10}{100} + \frac{400}{40.000}} \rceil = \lceil 9,09 \rceil = 10$$

$$B(Eventos \bowtie Apuestas) = \frac{n(Eventos \bowtie Apuestas)}{F(Eventos \bowtie Apuestas)} = \frac{400}{10} = 40$$

Denominando $PrimeraJunta = Eventos \bowtie Apuestas$. Busco el costo de $PrimeraJunta \bowtie Usuarios$, como PrimeraJunta lo mando por pipelining no tengo el costo de sumar su bloque

$$Costo(PrimeraJunta \bowtie Usuarios) = \lceil \frac{B(PrimeraJunta)}{M - 2} \rceil \cdot B(Usuarios)$$

$$Costo(PrimeraJunta \bowtie Usuarios) = \lceil \frac{40}{1.000 - 2} \rceil \cdot 50 = \lceil 50 \rceil = 50$$

Calculo ahora la cantidad de filas:

$$n(PrimeraJunta \bowtie Usuarios) = \frac{n(PrimeraJunta) \cdot n(Usuarios)}{\text{máx}(V(id, Eventos), V(id, Usuarios))}$$

$$n(PrimeraJunta \bowtie Usuarios) = \frac{400 \cdot 2.000}{\text{máx}(400, 2.000)}$$

En el paso previo no se si hay una forma más elegante de hacerlo pero entiendo que si tengo 400 filas del join previo como MUCHO mi variabilidad va ser de 400, no hay forma de que sea más. Desconozco si se puede estimar de otra forma.

$$n(PrimeraJunta \bowtie Usuarios) = \frac{400 \cdot 2.000}{2.000} = 400$$

Entonces como respuesta final tengo que la cardinalidad es:

$$n(Eventos \bowtie Apuestas \bowtie Usuarios) = 400$$

Y mi costo

$$Cost(Eventos \bowtie Apuestas \bowtie Usuarios) = Cost(FileScan(Eventos)) + Cost(Eventos \bowtie Apuestas) + Cost((Eventos \bowtie Apuestas) \bowtie Usuarios)$$

$$Cost(Eventos \bowtie Apuestas \bowtie Usuarios) = 1.000 + 400 + 50 = 1.450$$

1.

- Partidos(id_partido, id_jug_1, sets_jug_1, id_jug2, sets_jug_2)
- Jugadores(id_jugador, nombre_con_apodo, fecha_inscripcion)

Escriba una consulta que devuelva los id de partidos en que el tenista Jose Luis “Batata” Clerc le ganó al tenista Juan Carlos “Papa” Noel

Versión I:

```

SELECT p.id_partido
FROM Partidos p
    INNER JOIN Jugadores j ON p.id_jug1 = j.id_jugador
    INNER JOIN Jugadores j2 ON p.id_jug2 = j2.id_jugador
WHERE j.nombre_con_apodo = 'Jose Luis "Batata" Clere' AND
    j2.nombre_con_apodo = 'Juan Carlos "Papa" Noel' AND
    p.sets_jug_1 > p.sets_jug_2

```

UNION

```

SELECT p.id_partido
FROM Partidos p
    INNER JOIN Jugadores j ON p.id_jug1 = j.id_jugador
    INNER JOIN Jugadores j2 ON p.id_jug2 = j2.id_jugador
WHERE j.nombre_con_apodo = 'Juan Carlos "Papa" Noel' AND
    j2.nombre_con_apodo = 'Jose Luis "Batata" Clere' AND
    p.sets_jug_1 < p.sets_jug_2

```

Versión II:

```

SELECT p.id_partido
FROM Partidos p
    INNER JOIN Jugadores j ON p.id_jug1 = j.id_jugador
    INNER JOIN Jugadores j2 ON p.id_jug2 = j2.id_jugador
WHERE (
    j.nombre_con_apodo = 'Jose Luis "Batata" Clere' AND
    j2.nombre_con_apodo = 'Juan Carlos "Papa" Noel' AND
    p.sets_jug_1 > p.sets_jug_2
)
OR (
    j.nombre_con_apodo = 'Juan Carlos "Papa" Noel' AND
    j2.nombre_con_apodo = 'Jose Luis "Batata" Clere' AND
    p.sets_jug_1 < p.sets_jug_2
)

```

2.

```

PedidosNavideños(
    dni_niño,
    nombre,
    apodo,
    puntaje_conducta,
    direccion,
    cod_juguete,
    nombre_juguete,
    nombre_tarjeta,
    direccion_entrega
)

```

3.

Me pide encontrar:

dos amigos de Nicolas que sean amigos entre ellos, que no sean amigos de un enemigo de Nicolas, y que el par de amigos maximice la confianza que les tiene

```

MATCH (amigo1:Persona)-[confianza1: AMIGO_DE]->(Nico:Persona{nombre:'Nicolás'}),
    (amigo2:Persona)-[confianza2: AMIGO_DE]->(Nico)
WHERE amigo1.nombre < amigo2.nombre
WHERE NOT EXISTS {
    (amigo1)-[:AMIGO_DE]->(:Persona)-[:ENEMIGO_DE]->(Nico)
} AND NOT EXISTS {
    (amigo2)-[:AMIGO_DE]->(:Persona)-[:ENEMIGO_DE]->(Nico)
}
WITH amigo1, amigo2, confianza1.confianza + confianza2.confianza AS confianza_maxima

```

```

RETURN amigo1.nombre, amigo2.nombre, confianza_maxima
ORDER BY confianza_maxima DESC
LIMIT 1

```

4.

Realizar una consulta en el que se encuentren los regalos que hayan sido pedidos por al menos 10 hermanos de Argentina, considerando como hermanos a dos niños que viven en la misma dirección

```

db.regalos.aggregate([
  {
    $match:
      {
        pais: "Argentina"
      }
  },
  {
    $unwind: "$Regalos"
  },
  // asumo que existen direcciones iguales pero en distintas ciudades, y que existen direcciones y ciudades ig

  // busco cantidad de personas que pidieron en la misma dirección
  {
    $group: {
      _id: {
        regalo: "$Regalos",
        direccion: "$direccion",
        ciudad: "$ciudad"
      }
      cantidad_hermanos: {$sum: 1}
    }
  },
  // podria buscar si es mayor igual pero como habla de PAR de hermanos, lo dejo así
  {
    $match:
      {
        cantidad_hermanos: {$eq: 2}
      }
  },
  // agrupo por regalos y busco la cantidad de hermanos que lo pidieron (realmente la cantidad de personas que
  {
    $group: {
      _id: {
        regalo: "$_id.regalo",
      },
      total_hermanos: {$sum: "$cantidad_hermanos"}
    }
  },
  // por lo de arriba aca miro que sea mayor igual a 20, y no a 10
  {
    $match:
      {
        total_hermanos: { $gte: 20 }
      }
  },
  // proyecto mi respuesta final
  {
    $project: {
      _id: 0,
      regalo: "$_id.regalo"
    }
  }
])

```


5.

```
01 (BEGIN, T1);
02 (WRITE T1, A, 1, 2);
03 (COMMIT, T1);
04 (BEGIN, T2);
05 (WRITE T2, A, 2, 3);
06 (BEGIN CKPT, T2);
07 (BEGIN, T3);
08 (BEGIN, T4);
09 (WRITE T3, B, 4, 5);
10 (END CKPT);
11 (WRITE T3, C, 6, 7);
12 (COMMIT, T3);
13 (WRITE T4, C, 7, 8);
```

Miro que transacciones commitearon y cuales no primero:

Commitearon: T1, T3

No commitearon: T2, T4

Hago UNDO de las transacciones que no commitearon

C <- 7

Como veo el en ENDCKPT se que todo lo previo al BEGIN CKPT (06) está guardado en disco, por lo cual no tengo que deshacer eso.

Ahora hago REDO de las transacciones que commitearon.

Como tengo el CKPT en 06 T1 ya se que no está activa (commiteó). Empiezo a rehacer desde ahí

B <- 5

C <- 7

Finalmente escribo

(ABORT, T2)

(ABORT, T4)

Flusheo todo a disco, el log y los datos

6.

- Criaturitas(id_criaturita, nombre_criaturita, apellido, direccion, ciudad)
- Renos(id_reno, nombre_reno, ciudad)

Se efectua el join natural entre Criaturitas y Renos utilizando el atributo *ciudad*

Costo por Hash Grace y estimar filas y bloques.

Arranco por el costo

$$Cost(Criaturitas \bowtie Renos) = 3 \cdot (B(Criaturitas) + B(Renos)) = 3 \cdot (100 + 5.000) = 15.300$$

Para estimar la cantidad de filas lo que tengo que hacer es fijarme en el histograma primero cuantos valores coinciden con cuantos

$$n(Criaturitas \bowtie Renos) = n(Criaturitas_{conocidas} \bowtie Renos_{conocidas}) + n(Criaturitas_{estimadas} \bowtie Renos_{estimadas})$$

Calculo lo conocido primero entonces.

$$n(Criaturitas_{conocidas} \bowtie Renos_{conocidas}) = 10.000 \cdot 100 + 5.000 \cdot 50 + 5.000 \cdot 50 = 1.500.000$$

Ahora debo estimar la cantidad de filas restantes de Criaturitas que se unen con los Renos. Como solo tengo el valor de la Variabilidad de ambos, y puedo saber las filas restantes, asumo distribución uniforme de cada tabla entre el resto de valores.

$$n(Criaturitas_{restantes}) = 50.000 - 10.000 - 5.000 - 5.000 = 30.000$$

$$n(Renos_{restantes}) = 1.000 - 100 - 50 - 50 = 800$$

Ahora sí uso la formula de join normal

$$n(Criaturitas_{restantes} \bowtie Renos_{restantes}) = \frac{n(Criaturitas_{restantes}) \cdot n(Renos_{restantes})}{\max(V(ciudad, Criaturitas), V(ciudad, Renos))}$$

$$n(Criaturitas_{restantes} \bowtie Renos_{restantes}) = \frac{24.000.000}{\max(500, 300)} = \frac{24.000.000}{500} = 48.000$$

Concluyo entonces que la cantidad de filas estimada de la junta es:

$$n(Criaturitas \bowtie Renos) = n(Criaturitas_{conocidas} \bowtie Renos_{conocidas}) + n(Criaturitas_{estimadas} \bowtie Renos_{estimadas})$$

$$n(Criaturitas \bowtie Renos) = 1.500.000 + 48.000 = 1.548.000$$

Calculo ahora la cantidad de bloques:

$$F(Criaturitas \bowtie Renos) = \lfloor \frac{1}{\frac{1}{F(Criaturitas)} + \frac{1}{F(Renos)}} \rfloor$$

$$F(Criaturitas \bowtie Renos) = \lfloor \frac{1}{\frac{5.000}{50.000} + \frac{100}{1.000}} \rfloor = \lfloor 5 \rfloor = 5$$

$$B(Criaturitas \bowtie Renos) = \frac{n(Criaturitas \bowtie Renos)}{F(Criaturitas \bowtie Renos)} = \frac{1.548.000}{5} = 309.600$$

1.

Ganadores(nombre_torneo, año, ganador)

Escriba una consulta que devuelva a quienes hayan ganado al menos una vez el torneo “Champions Liga” pero nunca hayan ganado ni el torneo “Ke Torneo” ni el torneo “Torneo en Bloque”

Opción I:

```
SELECT g.ganador
FROM Ganadores g
WHERE g.nombre_torneo = 'Champions Liga'
```

EXCEPT

```
SELECT g.ganador
FROM Ganadores g
WHERE g.nombre_torneo = 'Ke Torneo' OR g.nombre_torneo = 'Torneo en Bloque'
```

Opción II:

```
SELECT g.ganador
FROM Ganadores g
WHERE g.nombre_torneo = 'Champions Liga'
AND g.ganador NOT IN (
    SELECT g2.ganador
    FROM Ganadores g2
    WHERE g2.nombre_torneo = 'Ke Torneo' OR g2.nombre_torneo = 'Torneo en Bloque'
)
```

2.

Escriba una consulta en Cypher que devuelva a todos los primos hermanos de una persona con DNI 12345678

```

MATCH (p1:Persona{DNI: 12345678}),
      (AbuelxP1:Persona)<-[:HIJO_DE]-(XadreP1:Persona)<-[:HIJO_DE]-(p1),
      (AbuelxP1)<-[:HIJO_DE]-(Tix:Persona)<-[:HIJO_DE]-(primx:Persona)
WHERE XadreP1.DNI <> Tix.DNI
RETURN primx

```

3.

Transaccion con $TS = 8$.

$$READ_TS(X) = 9$$

$$WRITE_TS(X) = 5$$

Puede leer: el $WRITE_TS$ es menor a TS , asique puede leer. En este caso actualizo el $READ_TS(X) = 8$. $WRITE_TS(X)$ queda igual.

Puede escribir: Como el $READ_TS$ es mayor a TS , no puede escribir.

4.

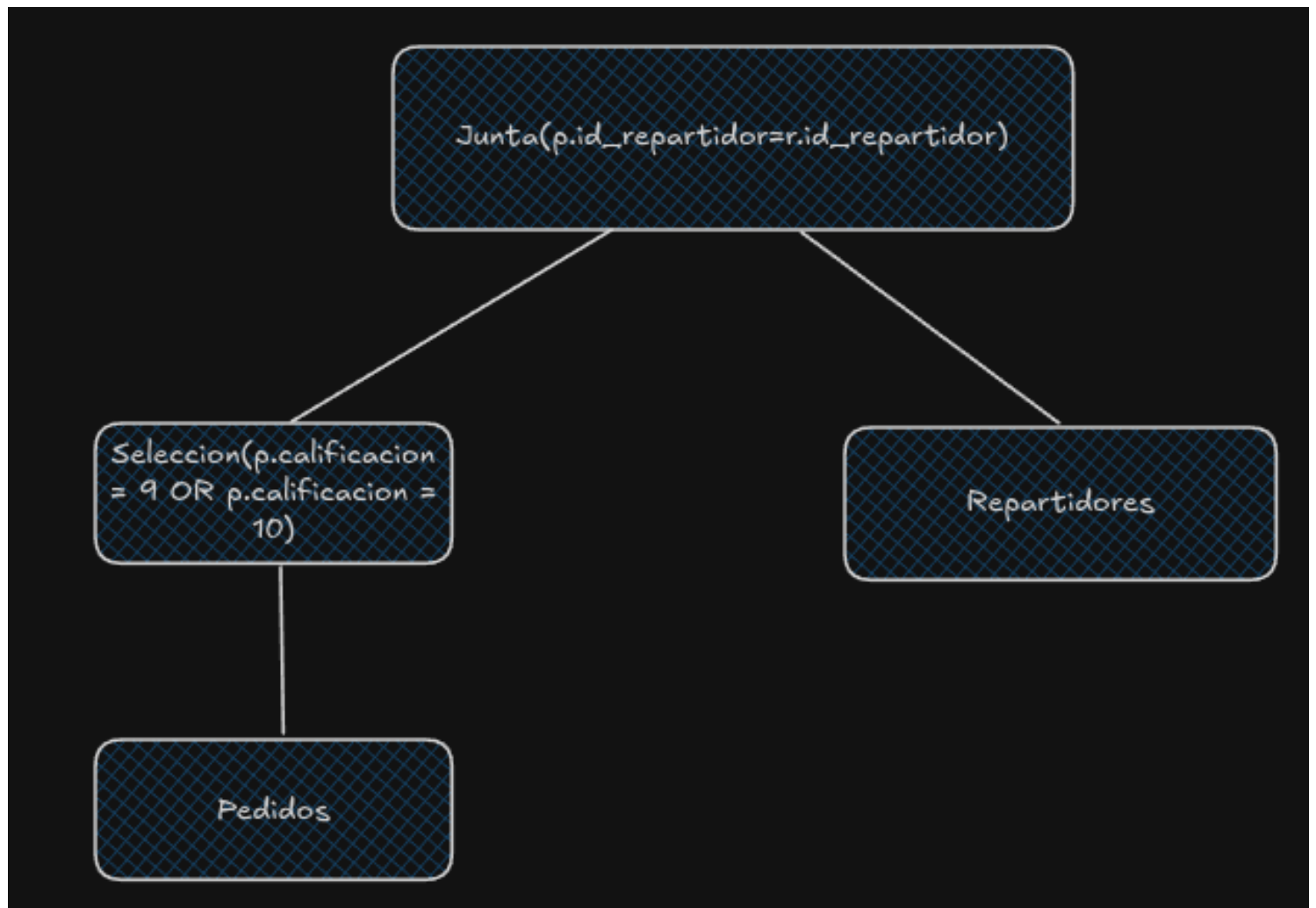


Figure 8: Árbol de consultas

Primero hago la selección para aprovechar pipelining después, como no hay índices, tengo que hacer un FileScan:

$$Cost(\sigma_{calificacion=9 \text{ OR } calificacion=10}(Pedidos)) = B(Pedidos) = 100.000$$

Calculo las filas que salen de aca:

$$n(\sigma_{calificacion=9 \text{ OR } calificacion=10}(Pedidos)) = 100.000 \cdot \frac{2}{8} = 25.000$$

Estimacion cantidad de bloques de la selección

$$B(\sigma_{calificacion=9 \text{ OR } calificacion=10}(Pedidos)) = \lceil \frac{n(\sigma_{calificacion=9 \text{ OR } calificacion=10}(Pedidos))}{F(R)} \rceil$$

$$B(\sigma_{calificacion=9 \text{ OR } calificacion=10}(Pedidos)) = \lceil \frac{25.000}{\frac{1.000.000}{100.000}} \rceil = \lceil 2.500 \rceil = 2.500$$

Calculo el costo de la junta ahora:

$$Cost(Pedidos_{\sigma} \bowtie Repartidores) = \lceil \frac{B_{\sigma}}{M-2} \rceil \cdot B(Repartidores)$$

$$Cost(Pedidos_{\sigma} \bowtie Repartidores) = \lceil \frac{2.500}{98} \rceil \cdot 2.000 = \lceil 25,51 \rceil \cdot 2.000 = 52.000$$

Tengo como costo total:

$$Cost(Pedidos \bowtie Repartidores) = Cost(FileScan(Pedidos)) + Cost(Pedidos_{\sigma} \bowtie Repartidores) = 100.000 + 52.000 = 152.000$$

5.

```
01 (BEGIN, T1);
02 (WRITE T1, A, 1, __ );
03 (BEGIN CKPT, ___ );
04 (COMMIT, T1);
05 (BEGIN, T2);
06 (WRITE T2, A, 2, 4);
07 (COMMIT, T2);
08 (END CKPT);
09 (BEGIN, T3);
10 (BEGIN CKPT, ___ );
11 (WRITE T3, A, ___ , 3);
12 (WRITE T3, B, 6, 7);
```

Completando los valores:

```
01 (BEGIN, T1);
02 (WRITE T1, A, 1, 2);
03 (BEGIN CKPT, T1);
04 (COMMIT, T1);
05 (BEGIN, T2);
06 (WRITE T2, A, 2, 4);
07 (COMMIT, T2);
08 (END CKPT);
09 (BEGIN, T3);
10 (BEGIN CKPT, T3);
11 (WRITE T3, A, 4, 3);
12 (WRITE T3, B, 6, 7);
```

Primero busco transacciones commiteadas y no commiteadas:

Commiteadas: T_1, T_2

No commiteadas: T_3

Hago UNDO de las transacciones no commiteadas:

B <- 6

A <- 4

Ahora hago REDO de las transacciones commiteadas de la transacción más vieja del BEGIN CKPT inicial

A <- 2

A <- 4

Escribo al final del LOG

(ABORT, T3)

Y flusheo todo a disco

6.

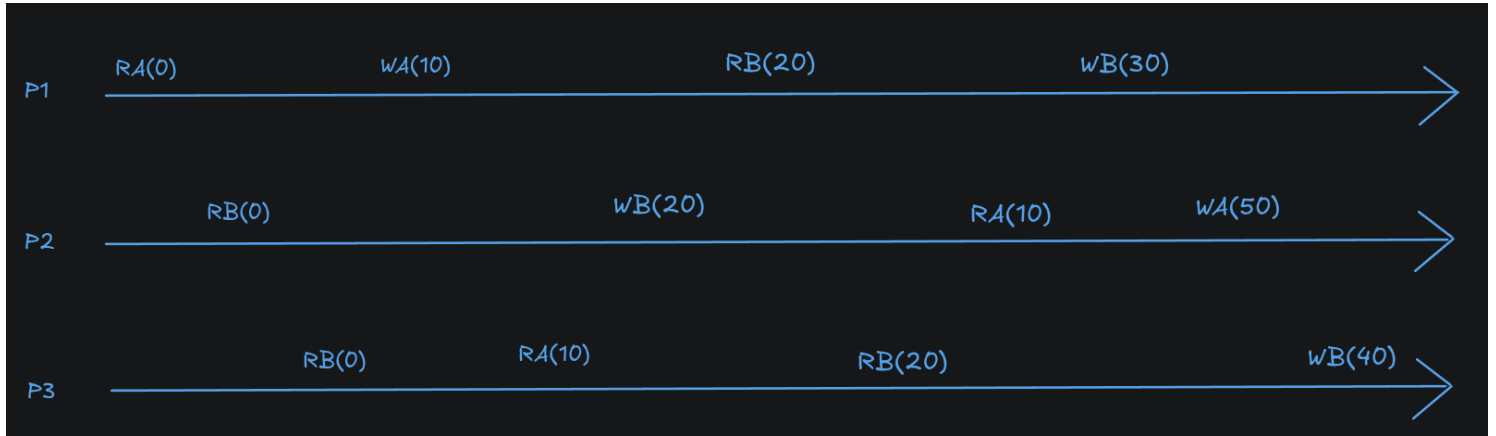


Figure 9: Consistencia secuencial

Concluyo entonces que esta ejecución tiene consistencia secuencial.

1.

Estudiantes(padron, nombre, apellido, provincia_nacimiento)

Escriba una consulta que devuelva aquellos apellidos que son tenidos por al menos un estudiante en todas las provincias de nacimiento.

```
SELECT e.apellido
FROM Estudiantes e
GROUP BY e.apellido
HAVING COUNT(DISTINCT e.provincia_nacimiento) = (
    SELECT COUNT(DISTINCT provincia_nacimiento)
    FROM Estudiantes
)
```

2.

Escriba una consulta que devuelva para cada nota de aprobación del parcial (se aprueba con 4 o más), la cantidad de alumnos que la sacaron

Opción I:

```
db.notas.aggregate([
  // separo segun notas de parcial
  {
    $unwind: "$notas_parcial"
  },
  // si una persona no aprobó la materia, simplemente no voy a encontrar ningun registro que sea con su nombre
  {
    $match: {
      "notas_parcial.nota": { $gte: 4 }
    }
  },
  {
    $group: {
      _id: "$notas_parcial.nota",
```

```

        cantidad_alumnos: {$sum: 1}
    },
    {
        $project: {
            _id: 0,
            nota: "$_id",
            cantidad_alumnos: 1
        }
    }
]
])

```

Opción II: (me quedo con la mayor de sus notas de aprobación)

```

db.notas.aggregate([
    // separo segun notas de parcial
    {
        $unwind: "$notas_parcial"
    },
    // si una persona no aprobó la materia, simplemente no voy a encontrar ningun registro que sea con su nombre
    {
        $match: {
            "notas_parcial.nota": {$gte: 4}
        }
    },
    {
        $group: {
            _id: "$padron",
            mejor_nota: {$max: "$notas_parcial.nota"}
        }
    },
    {
        $group: {
            _id: "$mejor_nota",
            cantidad_alumnos: {$sum: 1}
        }
    },
    {
        $project: {
            _id: 0,
            nota: "$_id",
            cantidad_alumnos: 1
        }
    }
]
])

```

3.

Plato: nombre, tipo Restaurante: nombre, direccion Ingrediente: nombre, tieneGluten

(p)-[:SE_VENDE_EN {precio}]->(r)

(p)-[:USA]->(i1)

Escriba una consulta que devuelva todas las combinaciones de restaurante, plato, principal y postre que sean apto para celíaco y que su costo total no superen los 9000 pesos.

```

MATCH(principal:Plato{tipo:"Principal"})-[sv1:SE_VENDE_EN]->(r1:Restaurante),
    (postre:Plato{tipo:"Postre"})-[sv2:SE_VENDE_EN]-(r1)
WHERE NOT EXISTS {
    (principal)-[u:USA]->(:Ingrediente{tieneGluten:true})
} AND NOT EXISTS {
    (postre)-[:USA]->(:Ingrediente{tieneGluten:true})
}
WITH principal, postre, r1, sv1.precio + sv2.precio AS precio_menu

```


Primero que nada realizo la selección para aprovechar el pipelining

Como no poseo índices voy a tener que cargar todos los bloques de la tabla compositores para hacer un FileScan

$$Cost(\sigma_{\text{año_fallecimiento} \leq 1800}(\text{Compositores})) = B(\text{Compositores})$$

Según los datos del enunciado ya se que solo el 10% de la muestra falleció **antes** de 1800.

$$n(\sigma_{\text{año_fallecimiento} \leq 1800}(\text{Compositores})) = 0,1 \cdot n(\text{Compositores}) = 0,1 \cdot 100.000 = 10.000$$

Calculo los bloques ahora:

Primero calculo el $F(\text{Compositores})$:

$$F(\text{Compositores}) = \frac{n(\text{Compositores})}{B(\text{Compositores})} = \frac{100.000}{20.000} = 5$$

$$B(\sigma_{\text{año_fallecimiento} \leq 1800}(\text{Compositores})) = \lceil \frac{n(\sigma_{\text{año_fallecimiento} \leq 1800}(\text{Compositores}))}{F(\text{Compositores})} \rceil = \lceil \frac{10.000}{5} \rceil = \lceil 2.000 \rceil = 2.000$$

Por la variabilidad, y mismo por el nombre de los campos, me doy cuenta que `id_compositor` es PK de la tabla `Compositores`, por lo cual me va quedar una variabilidad de 10.000 luego de la selección.

Ahora sí realizo la junta, pruebo con loops anidados ya que tengo bastantes bloques de memoria.

$$Cost(\text{Compositores}_\sigma \bowtie \text{Obras}) = \lceil \frac{B(\text{Compositores}_\sigma)}{M - 2} \rceil \cdot B(\text{Obras})$$

$$Cost(\text{Compositores}_\sigma \bowtie \text{Obras}) = \lceil \frac{2.000}{2.003} \rceil \cdot 500.000 = 500.000$$

Viendo que el costo de la junta es de solo $B(\text{Obras})$ (no sumo los bloques iniciales de compositores pues usé pipelining) $\rightarrow$ este es el mejor método, quedandome como costo final

$$Cost(\text{Compositores} \bowtie \text{Obras}) = Cost(\sigma_{\text{año_fallecimiento} \leq 1800}(\text{Compositores})) + Cost(\text{Compositores}_\sigma \bowtie \text{Obras})$$

$$Cost(\text{Compositores} \bowtie \text{Obras}) = 20.000 + 500.000 = 520.000$$

1.

Resultados(año, legajo, nombre, apellido, puntaje, puesto)

Escriba una consulta que devuelva nombre y apellido de aquellos integrantes que participando en mas de un año siempre hayan salido en el mismo puesto. Considerar que puede haber distintos integrantes con mismo nombre pero no con mismo legajo.

```
SELECT nombre, apellido
FROM (
    SELECT legajo,
           nombre,
           apellido
    FROM Resultados
    GROUP BY legajo
    HAVING COUNT(DISTINCT año) >= 1 AND COUNT(DISTINCT puesto) = 1
)
```

2.

UserTags ya ordenada por idTag. Join por idTag

$$M = 10.000$$

Ya a ojo veo que cumple con la restricción de memoria.

El costo de SortMerge es:

$$Cost(UserTags \bowtie PaperTags) = Cost(Ord_M(UserTags)) + Cost(Ord_M(PaperTags)) + B(UserTags) + B(PaperTags)$$

Como la tabla UserTags ya la tengo ordenada, el costo de esa parte es 0.

Calculo el costo de ordenar PaperTags

$$Cost(Ord_M(PaperTags)) = 2 \cdot B(PaperTags) \cdot \lceil \log_{M-1}(B(PaperTags)) \rceil$$

$$Cost(Ord_M(PaperTags)) = 2 \cdot 9.000 \cdot \lceil 0,98 \rceil = 18.000$$

Vuelvo a la formula del costo total entonces:

$$Cost(UserTags \bowtie PaperTags) = Cost(Ord_M(UserTags)) + Cost(Ord_M(PaperTags)) + B(UserTags) + B(PaperTags)$$

$$Cost(UserTags \bowtie PaperTags) = 0 + 18.000 + 20.000 + 9.000 = 47.000$$

Cantidad de filas:

$$n(UserTags \bowtie PaperTags) = \frac{n(UserTags) \cdot n(PaperTags)}{\max(V(idTag, UserTags), V(idTag, PaperTags))}$$

$$n(UserTags \bowtie PaperTags) = \frac{2.000.000 \cdot 900.000}{\max(80, 100)} = \frac{2.000.000 \cdot 900.000}{100}$$

$$n(UserTags \bowtie PaperTags) = \frac{1.800.000.000.000}{100} = 18.000.000.000$$

Estimo la cantidad de bloques ahora:

$$F(UserTags) = \frac{2.000.000}{20.000} = 100$$

$$F(PaperTags) = \frac{900.000}{9.000} = 100$$

Estimo factor de bloques

$$F(UserTags \bowtie PaperTags) = \lfloor \frac{1}{\frac{1}{F(UserTags)} + \frac{1}{F(PaperTags)}} \rfloor =$$

$$F(UserTags \bowtie PaperTags) = \lfloor 50 \rfloor = 50$$

$$B(UserTags \bowtie PaperTags) = \frac{18.000.000.000}{50} = 360.000.000$$

3.

Obtener para cada persona que viajo al menos 5000km en total, la cantidad de viajes que realizó y la cantidad de kilometros total recorridos.

```

db.vuelos.aggregate([
  {
    $unwind: "$pasajeros"
  },
  {
    $group: {
      _id: "$pasajeros.dni",
      nombre: { $first: "$pasajeros.nombre" },
      km_recorridos: { $sum: "$distancia_km" },
      cantidad_viajes: { $sum: 1 }
    }
  },
  {
    $match: {
      km_recorridos: { $gte: 5000 }
    }
  },
  {
    $project: {
      _id: 0,
      dni_pasajero: "$pasajeros.dni",
      nombre_pasajero: "$pasajeros.nombre",
      km_recorridos: 1,
      cantidad_viajes: 1
    }
  }
])

```

4.

N1: 155
 N2: 235
 N3: 120
 N4: 60

a.

I1: N4
 I2: N3
 I3: N1

- b. En caso de que se caiga N3 aquellos datos guardados en N3 deben pasar a N1, y los valores de resguardo (que pertenecian a N3 pero que estaban salvados en N1) pasan a estar en N2.

5.

```

01 (BEGIN, T1)
02 (WRITE T1, A, _)
03 (BEGIN, T2)
04 (WRITE T1, B, _)
05 (BEGIN CKPT, T1, T2)
06 (COMMIT T1)
07 (WRITE T2, A, _)
08 (WRITE T2, C, _)
09 (END CKPT)
10 (BEGIN, T3)
11 (WRITE T3, B, _)
12 (COMMIT, T3)
13 (COMMIT, T2)

```

Llenando el log con los datos

```

01 (BEGIN, T1)
02 (WRITE T1, A, 10)
03 (BEGIN, T2)
04 (WRITE T1, B, 20)

```

```

05 (BEGIN CKPT, T1, T2)
06 (COMMIT T1)
07 (WRITE T2, A, 30)
08 (WRITE T2, C, 40)
09 (END CKPT)
10 (BEGIN, T3)
11 (WRITE T3, B, 50)
12 (COMMIT, T3)
13 (COMMIT, T2)

```

Esto es así porque el algoritmo es REDO, por lo cual escribo los V_{new}

Para la etapa de recuperación debo rehacer las transacciones ya commiteadas. Tengo un CKPT pero no me sirve de nada porque no hay ninguna transacción commiteada previa al BEGIN. Arranco desde 02 entonces. PD: noto que todas las transacciones están commiteadas asique voy a rehacer todo

```

A <- 10
B <- 20

```

Piso A: A <- 30

```

C <- 40

```

Piso B: B <- 50

En el archivo de log no escribo nada porque todas las transacciones commitearon, y flusheo todo a disco.

6.

$$b_{T_1}; b_{T_2}; W_{T_1}(X); W_{T_2}(Y); R_{T_2}(X); W_{T_1}(Z)$$

Para que el solapamiento cumpla con 2PL una transacción no puede adquirir un lock luego haber liberado otro. Propongo

$$b_{T_1}; b_{T_2}; L_{T_1}(X); L_{T_1}(Z); W_{T_1}(X); U_{T_1}(X); L_{T_2}(Y); L_{T_2}(X); W_{T_2}(Y); R_{T_2}(X); U_{T_2}(Y); U_{T_2}(X); W_{T_1}(Z); U_{T_1}(Z)$$

En el segundo item entiendo que al no tener los commits no puedo mostrar cuando libero los locks; aun pudiendo ubicarlos donde quisiera, no podria hacerlo porque sí o sí dsp de W_{T_1} tengo que liberar a $T_1(X)$ y entonces deberia hacer un commit ahi, pero dsp tengo una operación más que realiza T_1 (la última listada), asique no voy a poder cumplir con S2LP

Para que el solapamiento **no sea recuperable** lo que tengo que hacer es que aquellas transacciones que **leen** elementos que fueron modificados previamente por otras transacciones, commiteen **antes** que las transacciones modificadoras.

Propongo entonces agregar al final $C_{T_2}; C_{T_1}$

1.

Escriba una consulta que recomiende dos Mokepones a utilizar por Juan para batallar contra otro jugador (parametrizado como de nombre 'x') buscando aquellos dos Mokepones con mayor poder de ataque combinado entre ellos, y para los que no haya un Mokepon del otro jugador que le genere desventaja a Juan.

```

MATCH(juan:Jugador{nombre:'Juan', edad: 45})-[:POSEE]->(mj1:Mokepon)-[:ES_DE]->(e1:Elemento),
(e1)-[:DESVENTAJA_CONTRA]->(edv1:Elemento),
(juan)-[:POSEE]->(mj2:Mokepon)-[:ES_DE]->(e2:Elemento),
(e2)-[:DESVENTAJA_CONTRA]->(edv2:Elemento)
WHERE mj1.nombre > mj2.nombre AND NOT EXISTS {
  MATCH(:Jugador{nombre: 'x'})-[:POSEE]->(mjrival:Mokepon)-[:ES_DE]->(erival:Elemento),
  (e1)-[:DESVENTAJA_CONTRA]->(erival)
} AND NOT EXISTS {
  MATCH(:Jugador{nombre: 'x'})-[:POSEE]->(mjrival:Mokepon)-[:ES_DE]->(erival:Elemento),
  (e2)-[:DESVENTAJA_CONTRA]->(erival)
}
WITH mj1, mj2, mj1.ataque + mj2.ataque AS ataque_combinado
RETURN mj1, mj2, ataque_combinado
ORDER BY ataque_combinado DESC
LIMIT 1

```

2.

NotasAprobacion(padron, codigo, numero, fecha, nota)

Escribir una consulta que devuelve el padron de los alumnos que en todas sus materias tienen una nota de aprobacion mejor que el promedio de notas de la materia.

Opción I:

```
SELECT
    n.padron
FROM
    Notasaprobacion n
GROUP BY
    n.padron
HAVING
    COUNT(*) = (
        SELECT
            COUNT(*)
        FROM
            Notasaprobacion n2
        WHERE
            n2.padron = n.padron
        AND
            n2.nota >= (
                SELECT
                    AVG(n3.nota)
                FROM
                    Notasaprobacion n3
                WHERE
                    n3.codigo = n2.codigo AND
                    n3.numero = n2.numero
                GROUP BY
                    n3.codigo, n3.numero
            )
        )
    )
```

Opción II:

```
SELECT
    n.padron
FROM
    Notasaprobacion n
WHERE NOT EXISTS (
    SELECT
        1
    FROM
        Notasaprobacion n2
    WHERE
        n.padron = n2.padron
    AND n2.nota < (
        SELECT
            AVG(nota)
        FROM
            Notasaprobacion n3
        WHERE
            n3.codigo = n2.codigo
            AND n3.numero = n2.numero
        GROUP BY
            n3.codigo, n3.numero
        )
    )
    )
```

3.

Escriba una consulta que devuelva un histograma con la cantidad de cada puntaje que dejada cada usuario en sus reseñas

En este no estoy 100% seguro si entendi bien el enunciado, entiendo que se quiere buscar la cantidad de veces que cada usuario pone X puntaje, contando eso. Por ejemplo

```
{username: 'Tael', puntaje:5, cantidad_restos: 10}
{username: 'Tael', puntaje:3, cantidad_restos: 2}
{username: 'Lucas', puntaje:4, cantidad_restos: 8}
```

```
db.restoran.aggregate([
  {
    $unwind: "$reseñas"
  },
  {
    $group: {
      _id: {
        username: "$reseñas.username",
        puntaje: "$reseñas.puntaje"
      },
      cantidad_restos: {$sum: 1}
    }
  },
  {
    $project: {
      _id: 0,
      username: "$_id.username",
      puntaje: "$_id.puntaje",
      cantidad_restos: 1
    }
  }
])
```

4.

Hoteles(id_hotel, nombre, estrellas, provincia, ciudad, dirección, contacto)

SELECT * FROM Hoteles WHERE ciudad = 'Bella Vista';

- a. Indice(provincia, ciudad): Entiendo que en este caso no me sirve de nada el índice ya que deberia primero buscar por provincia y recién ahí utilizar la ciudad. El costo de esta consulta sería hacer un FileScan

$$Cost(\sigma_{Ciudad='BellaVista'}(Hoteles)) = Cost(FileScan(Hoteles)) = B(Hoteles) = 900.000$$

- b. Indice(ciudad, provincia): En este caso entiendo que sí puedo buscar por índice, así que busco

$$Cost(\sigma_{Ciudad='BellaVista'}(Hoteles)) = Height(I(Ciudad, Hoteles)) + \lceil \frac{n(Hoteles)}{V(Ciudad, Hoteles)} \rceil = 4 + \lceil \frac{2.000.000}{500} \rceil = 4 + \lceil 4.000 \rceil = 4.004$$

c.

$$n(\sigma_{Ciudad='BellaVista'}(Hoteles)) = \lceil \frac{n(Hoteles)}{V(Ciudad, Hoteles)} \rceil = \lceil \frac{2.000.000}{500} \rceil$$

$$n(\sigma_{Ciudad='BellaVista'}(Hoteles)) = 4.000$$

5.

Transaccion con $TS = 91$. Indique un par de valores $READ_{TS}(X); WRITE_{TS}(X)$ para los que pueda ocurrir las siguientes situaciones.

- a. Puede leer pero no escribir

$$READ_{TS}(X) = 92; WRITE_{TS}(X) = 90$$

// puede leer porque ninguna transacción posterior modificó el item

- b. Puede escribir pero no leerlo.

Para que una transacción pueda escribir sobre un ítem, tanto $READ_{TS}(X)$ como $WRITE_{TS}(X)$ tienen que ser menores al TS de la transacción. Pero a su vez para que NO pueda leerlo el $WRITE_{TS}(X) \geq TS$ lo cual forma una contradicción, haciendo imposible esta situación.

c. No puede ni leer ni escribir X.

$READ_{TS}(X) = 90$; $WRITE_{TS}(X) = 92$

// no puede leer por que una transacción posterior lo modificó, y no puede modificarlo por la misma razón

6.

```
01 (BEGIN, T1)
02 (WRITE T1, A, _)
03 (BEGIN, T2)
04 (WRITE T1, B, _)
05 (BEGIN CKPT, T1, T2)
06 (COMMIT, T1)
07 (WRITE T2, A, _)
08 (WRITE T2, C, _)
09 (BEGIN, T3)
10 (COMMIT, T2)
11 (WRITE T3, C, _)
12 (COMMIT, T3)
```

Como es el algoritmo REDO, lo que se escribe en el log es el V_{new}

Nos queda entonces:

```
01 (BEGIN, T1)
02 (WRITE T1, A, 4)
03 (BEGIN, T2)
04 (WRITE T1, B, 5)
05 (BEGIN CKPT, T1, T2)
06 (COMMIT, T1)
07 (WRITE T2, A, 6)
08 (WRITE T2, C, 7)
09 (BEGIN, T3)
10 (COMMIT, T2)
11 (WRITE T3, C, 8)
12 (COMMIT, T3)
```

De nuevo, como es el algoritmo REDO, se van a rehacer las transacciones que commitearon y abortar aquellas que no.

Tengo un CKPT sin su END CKPT, asique no me sirve de nada.

Arranco desde la linea 02, y veo también que todas las transacciones commitearon (in other words, tengo que rehacer todo)

```
A <- 4
B <- 5
```

```
Piso A: A <- 6
C <- 7
```

```
Piso C: C <- 8
```

En el archivo de log no debo hacer nada, flusho todo a disco.

7.
8.

Escriba una consulta que devuelva el nombre de las instituciones con alto grado de inmadurez. Esto se determina cuando hay mas de 20 personas pertenecientes a la institucion para los que mas del 50% de sus intereses están registrados como inmaduros.

```
MATCH(p:Persona)-[:PERTENECE_A]->(i:Institucion),
      (p)-[:INTERESADO_EN]->(interes:Interes)
WITH p, i, COUNT(DISTINCT interes) AS total_intereses, COUNT(CASE WHEN interes.inmaduro THEN 1 END) AS intereses_inmaduros
WHERE intereses_inmaduros > 0.5 * total_intereses
WITH p, i, COUNT(DISTINCT p.id) AS cantidad_personas
```

```
WHERE cantidad_personas > 20
RETURN i.nombre
```

2.

Devuelva los ids de usuarios que en 2025 visitaron las mismas ciudades que Johann Sebastian visitó en el mismo año. EL id del celular de Johann Sebastian es 12345. Los stalkers pueden haber visitado otras ciudades además que las de Johann

// asumo que puedo hacer fecha.año si no tendria que comparar con horas y demás que me da un toque de paja sinceramente.

$$visitas_2025 = \sigma_{fecha.año=2025}(Ubicacion)$$

$$ciudades_johan = \pi_{id_ciudad}(\sigma_{id_celular=12345}(visitas_2025))$$

$$ubicaciones_no_johan = \sigma_{id_celular <> 12345}(visitas_2025)$$

$$otras_personas = \pi_{id_celular}(ubicaciones_no_johan \bowtie_{ubicaciones_no_johan.id_ciudad=ciudades_johan.id_ciudad} ciudades_johan)$$

$$otras_personas_real = \pi_{id_celular}(otras_personas \div ciudades_johan)$$

3.

Genere una consulta que devuelva los 5 paises con la mayor cantidad de juegos en los que un usuario del pais consiguio al menos un logro

```
db.logros.aggregate([
  {
    $unwind: "$usuarios"
  },
  {
    $group: {
      _id: {
        pais: "$usuarios.pais",
        juego: "$juego"
      }
    }
  },
  {
    $group: {
      _id: "$_id.pais"
    },
    cantidad_juegos_con_logro: {$sum: 1}
  },
  {
    $sort: {
      juegos_con_logro: -1
    }
  },
  {
    $limit: 5
  }
])
```

4.

```
01 (BEGIN, T1)
02 (WRITE T1, A, 0, 1)
03 (BEGIN, T2)
04 (WRITE T1, B, 0, 2)
05 (BEGIN CKPT, T1, T2)
06 (COMMIT, T1)
07 (WRITE T2, A, 1, 3)
08 (WRITE T2, C, 0, 4)
09 (BEGIN, T3)
10 (COMMIT, T2)
11 (WRITE T3, B, 2, 5)
```

Como no hay un end ckpt, tengo que ir hasta el inicio del log. Inicialmente tienen todos 0.

En disco entonces puedo tener:

A: 0; 1; 3.

B: 0; 2, 5.

C: 0; 4.

5.

Transaccion con $TS = 40$ intenta acceder a un item X. Indica para cada una de las siguientes situaciones si puede leer o no el item y si puede escribir o no el item.

a.

Lectura: puede ($WRITE_TS(X) \leq TS$)

Escritura: puede ($WRITE_TS(X) \leq TS$ AND $READ_TS(X) \leq TS$)

b.

Lectura: puede ($WRITE_TS(X) \leq TS$)

Escritura: no puede ($READ_TS(X) > TS$)

c.

Lectura: no puede ($WRITE_TS(X) > TS$)

Escritura: no puede ($WRITE_TS(X) > TS$)

d.

Lectura: no puede ($WRITE_TS(X) > TS$)

Escritura: no puede ($WRITE_TS(X) > TS$, ya con esto no puede pero ademas $READ_TS(X) > TS$ asique tampoco podría po

6.

SELECT *

FROM repartidores r

INNER JOIN pedidos p USING (id_repartidor)

WHERE p.calificacion = 10 OR p.calificacion = 9

Realizo primero la selección para aprovechar el pipelining. Como no hay indices, voy a tener que hacer un FileScan

$$Cost(\sigma_{calificacion=10 \vee calificacion=9}(Pedidos)) = Cost(FileScan(Pedidos)) = B(Pedidos) = 1.000.000$$

$$n(\sigma_{calificacion=10 \vee calificacion=9}(Pedidos)) = \frac{n(Pedidos)}{V(calificacion, Pedidos)} + \frac{n(Pedidos)}{V(calificacion, Pedidos)}$$

$$n(\sigma_{calificacion=10 \vee calificacion=9}(Pedidos)) = \frac{2}{V(calificacion, Pedidos)} \cdot n(Pedidos) = \frac{2}{5} \cdot 10.000.000 = 4.000.000$$

Estimo cantidad de bloques

$$F(Pedidos) = \frac{n(Pedidos)}{B(pedidos)} = \frac{10.000.000}{1.000.000} = 10$$

$$B(\sigma_{calificacion=10 \vee calificacion=9}(Pedidos)) = \lceil \frac{n(\sigma_{calificacion=10 \vee calificacion=9}(Pedidos))}{F(Pedidos)} \rceil$$

$$B(\sigma_{calificacion=10 \vee calificacion=9}(Pedidos)) = \lceil \frac{4.000.000}{10} \rceil = \lceil 400.000 \rceil = 400.000$$

Tengo $M = 200$ y me piden loops anidados por bloque, asique utilizo la formula del método

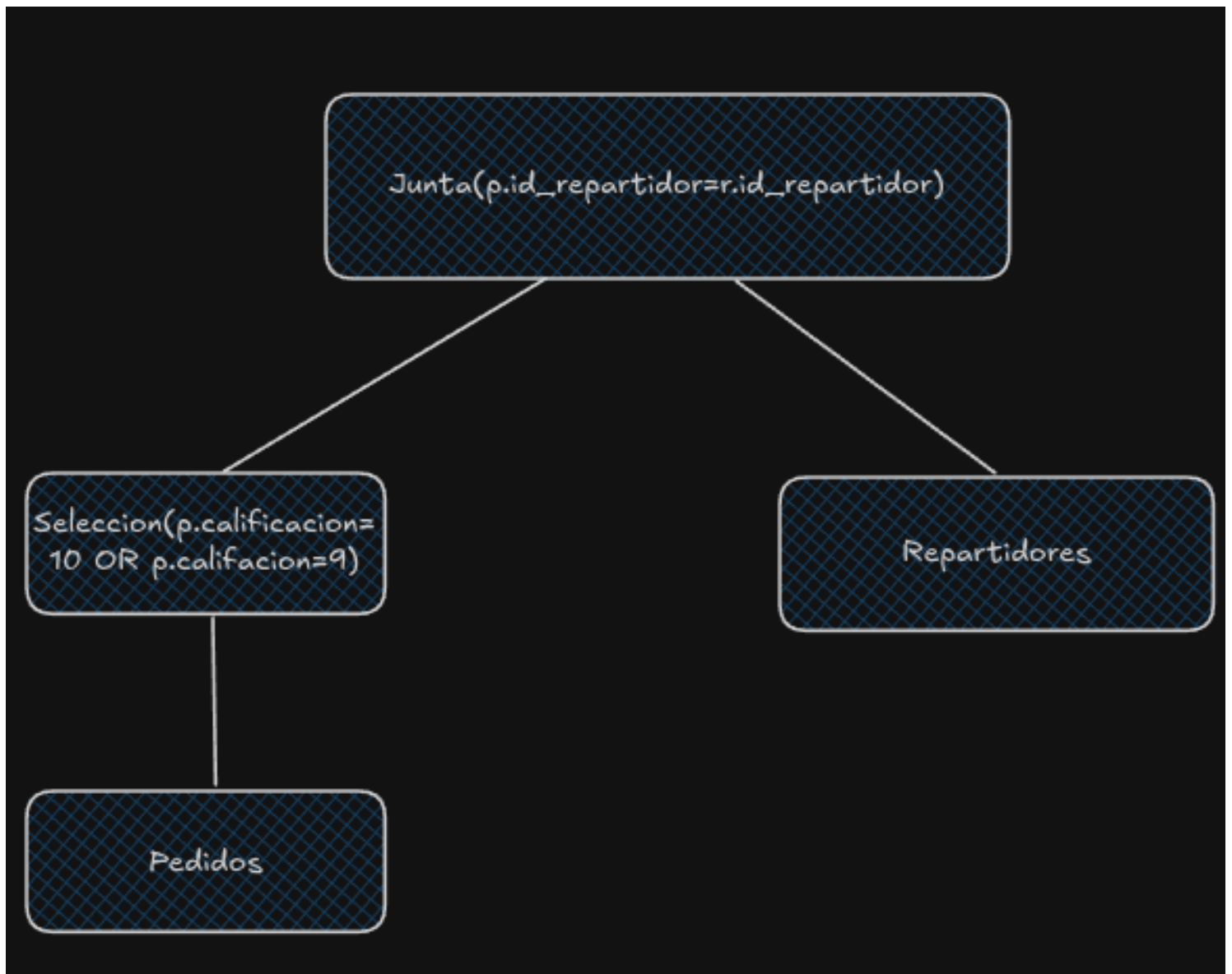


Figure 11: Árbol de consulta

$$Cost(Pedidos_{\sigma} \bowtie Repartidores) = \lceil \frac{B(Pedidos_{\sigma})}{M-2} \rceil \cdot B(Repartidores)$$

$$Cost(Pedidos_{\sigma} \bowtie Repartidores) = \lceil \frac{400.000}{198} \rceil \cdot 5.000 = \lceil 2.020, 2 \rceil \cdot 5.000 = 2.021 \cdot 5.000 = 10.105.000$$

Entonces el costo total de la consulta es

$$Cost(consulta) = Cost(\sigma_{calificacion=10 \vee calificacion=9}(Pedidos)) + Cost(Pedidos_{\sigma} \bowtie Repartidores)$$

$$Cost(consulta) = 1.000.000 + 10.105.000 = 11.105.000$$